

# 3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors  
Anonymous submission

## ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) strikes an appealing balance between rendering quality and efficiency across a broad range of scenes, yet its core reliance on approximating 3D Gaussians as 2D projections is an assumption that erodes accuracy whenever cameras depart from the idealized narrow-view regime—most visibly under the generic, large field-of-view (FoV) cameras that pervade real-world capture. Despite a growing body of extensions, no prior approach has simultaneously delivered both *projective exactness* and *real-time efficiency* across the diversity of camera models encountered in practice. We introduce 3DGEER, a geometrically exact and efficient Gaussian rendering framework designed to operate uniformly over this full spectrum of cameras. Working from first principles, we derive a closed-form expression for integrating Gaussian density along a ray, which enables precise forward rendering and differentiable optimization under essentially arbitrary camera models rather than any single hand-tuned projection. To retain efficiency across these settings, we propose the *Particle Bounding Frustum (PBF)*, which provides tight ray–Gaussian association without BVH traversal, together with the *Bipolar Equiangular Projection (BEAP)*, a representation that unifies disparate FoV models under a single parameterization, accelerates association, and improves reconstruction quality. Experiments spanning both pinhole and fisheye datasets show that 3DGEER outperforms prior methods across all metrics, runs  $5\times$  faster than existing projective-exact ray-based baselines, and generalizes to wider FoVs never seen during training—establishing a new state of the art for real-time radiance field rendering under diverse imaging conditions.

## 1 INTRODUCTION

Initiated by 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), the family of volumetric particle rendering methods formulated under splatting (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) has gained broad popularity across scene types and applications, owing to its impressive visual fidelity and high rendering speed. This efficiency stems fundamentally from EWA splatting (Zwicker et al., 2001), which uses a local affine approximation to project a 3D Gaussian as a 2D Gaussian on the image plane. While this approximation is effective across the narrow field-of-view (FoV) scenarios that dominate common benchmarks, it introduces substantial errors as soon as one moves toward the wider-FoV regime, where nonlinear distortions cause the true projection to deviate significantly from a symmetric 2D Gaussian—an effect that becomes acute in the fisheye imagery routinely captured in robotics, autonomous driving, and other embodied settings (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019, 2024; Sekkat et al., 2022). Although recent efforts have adapted differentiable rendering to particular fisheye camera models (Liao et al., 2024) or to broader families of camera models (Huang et al., 2024b), such approaches all remain fundamentally tethered to 2D projective-space approximations, and none has so far yielded an exact closed-form solution that holds uniformly across camera types.

Meanwhile, Celarek et al. (2025) questions the necessity of exact rendering by scaling up the number of Gaussians (up to 1M) to mitigate projective approximation in splatting. However, that evidence is confined to small object-centric scenes and does not obviously transfer to the wider range of settings encountered in practice. In large-scale scene reconstructions, the number of Gaussians required

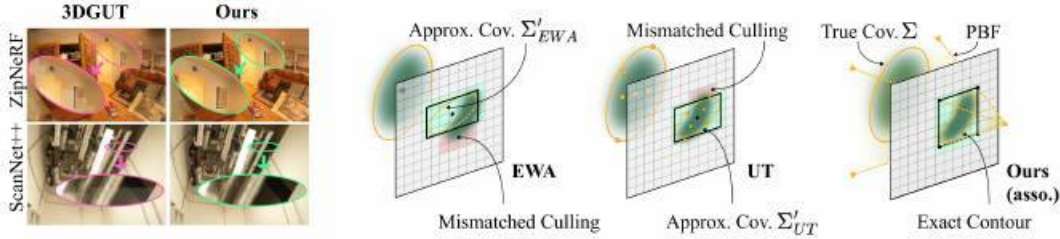


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts induced by inaccurate UT association. (Right) Diagram illustrating our association alongside representative alternatives. Across camera models, our method sidesteps the intermediate conic approximation by directly computing the exact bounding structure from the true 3D covariance.

varies dramatically across objects and frustums, so scaling this strategy up to larger and more diverse scenes becomes prohibitively costly. Beyond scene scale, we further show that projective errors grow substantially under generic distorted cameras, a regime in which simply increasing Gaussian counts cannot close the quality gap (see Fig. 5).

A parallel line of work instead tackles volumetric particle rendering directly in a ray-tracing/marching formulation (Yu et al. 2024b, Condor et al. 2025, Mai et al. 2025, Moënnelocoz et al. 2024, Talegaonkar et al. 2025, Gu et al. 2025). Because they operate in ray space, these approaches naturally accommodate large-FoV and generally distorted camera models and are, in principle, free from projective approximation error. Achieving high efficiency across such settings, however, remains a central challenge—especially in associating each rendering ray with its most relevant 3D particles. Considerable effort has therefore gone into accelerating this association, for instance through the advanced Bounding Volume Hierarchies (BVH) employed in EVER (Mai et al. 2025) and 3DGRT (Moënnelocoz et al. 2024). Even so, the inherent algorithmic complexity and limited parallelizability mean that pure ray-based methods have yet to match the efficiency of 3DGS across the workloads where splatting excels. As an alternative, the hybrid approach in 3DGUT (Wu et al. 2025) couples ray-tracing rendering with rasterization to solve ray-particle association efficiently. Yet the projective approximation error inherited from the association stage still compromises rendering exactness in projective geometry and can even induce grid-line artifacts (see Fig. 1 Left). To date, then, no single method has attained both the geometric exactness of ray-based rendering and the high efficiency of splatting simultaneously.

In this work, we propose 3DGEER—a 3D Gaussian Exact and Efficient Rendering method that achieves *exactness in projective geometry* (see definition in Sec. A) while retaining *high efficiency comparable to splatting*, and that does so uniformly for generic cameras spanning perspective projection through a wide variety of fisheye projection models. 3DGEER rests on three key contributions: (i) We revisit the heuristic maximum-response function (Keselman & Hebert 2022) of Gaussians from first principles and expose its connection to projective exactness. Building on this, we derive a differentiable rendering framework that is numerically stable and, by construction, independent of any particular camera projection. (ii) To perform efficient ray-particle association without sacrificing geometric exactness, we recast the problem as associating Camera Sub-frustums (CSF) with Particle Bounding Frustums (PBF)—analogous to the tile-AABB mapping used in 3DGS, but posed directly in 3D so that it applies across camera models. For each 3D Gaussian, we derive a closed-form solution for the PBF that tightly bounds it, enabling highly efficient association. (iii) In addition, we propose uniformly sampling rays in a novel Bipolar Equiangular Projection (BEAP) space for color supervision. This ray-sampling strategy not only aligns image-space partitioning with the underlying CSF but also improves reconstruction quality relative to conventional pinhole or equidistant projections across the scenes we study.

Extensive experiments spanning both fisheye and pinhole datasets—including ScanNet++ (Yeshwanth et al. 2023), Zip-NeRF (Barron et al. 2023), Aria (Lv et al. 2025), and MipNeRF (Barron et al. 2021), which together cover indoor and outdoor, object-centric and large-scale, and narrow-through wide-FoV capture—show that 3DGEER consistently outperforms prior and concurrent methods, achieving 0.9–4.8 PSNR gains under challenging wide-FoV conditions and 0.3–2.0 on pinhole views. Notably, our approach even surpasses splat-based methods on wide-FoV cameras when trained only with narrow-FoV data, underscoring a strong capacity to generalize beyond the imaging conditions seen at training time, all the way to extremely wide-FoV rendering.

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The top section groups splatting-based methods, while the bottom section presents ray-based approaches—several of which still fall back on projective-approximated association (e.g., EWA or the Unscented Transform). The comparison spans the axes that matter for general-purpose deployment, including camera FoV coverage, runtime, and exactness.

| Method                            | Render | Particle Asso. Method | Scope  | Particle Type | Large FoV | High FPS | Exact Meth. |
|-----------------------------------|--------|-----------------------|--------|---------------|-----------|----------|-------------|
| 3DGS [Kerbl et al. 2023]          | Splat. | EWA                   | Img.   | 3D Gauss.     | ✗         | ✓        | ✗           |
| FisheyeGS [Liao et al. 2024]      | Splat. | EWA                   | Img.   | 3D Gauss.     | ✓         | ✓        | ✗           |
| GS++ [Huang et al. 2024b]         | Splat. | EWA                   | Img.   | 3D Gauss.     | ✗         | ✓        | ✗           |
| 2DGS [Huang et al. 2024a]         | Ray    | Splat Bound.          | Img.   | 2D Gauss.     | ✗         | ✓        | ✓           |
| GOF [Yu et al. 2024b]             | Ray    | EWA                   | Img.   | 3D Gauss.     | ✗         | ✗        | ✗           |
| EVER [Mai et al. 2025]            | Ray    | BVH                   | Scene  | 3D Ellip.     | ✓         | ✗        | ✓           |
| 3DGRT [Moënné-Loccoz et al. 2024] | Ray    | BVH                   | Scene  | 3D Gauss.     | ✓         | ✗        | ✓           |
| 3DGUT [Wu et al. 2025]            | Ray    | UT                    | Img.   | 3D Gauss.     | ✓         | ✓        | ✗           |
| HTGS [Hahlbohm et al. 2025]       | Ray    | Splat Bound.          | Img.   | 3D Gauss.     | ✗         | ✓        | ✓           |
| 3DGEER (Ours)                     | Ray    | PBF                   | Frust. | 3D Gauss.     | ✓         | ✓        | ✓           |

## 2 RELATED WORKS

### 2.1 SPLATTING-BASED GAUSSIAN RENDERING

The large majority of 3DGS variants [Yu et al. 2024a, Zhang et al. 2024, Rota Bulò et al. 2024, Gao et al. 2025, Zhao et al. 2025, Kerbl et al. 2024, Hou et al. 2025, Liu et al. 2025] inherit the original EWA splatting strategy [Zwicker et al. 2001, Ren et al. 2002], projecting 3D Gaussians onto the image plane and approximating their appearance as 2D conics. To extend differentiable rendering toward large-FoV cameras with distortion, methods such as FisheyeGS [Liao et al. 2024] and GS++ [Huang et al. 2024b] estimate the Jacobian of equidistant or spherical projections in order to adjust the 2D mean and covariance of the projected Gaussians. Regardless of the specific projection targeted, however, these splatting-based approaches still rest on a first-order Taylor expansion of a highly non-linear projection function. Once the field of view widens, higher-order terms become non-negligible, producing significant approximation errors and degraded reconstruction quality.

More fundamentally, the true projection of a 3D Gaussian under any non-linear camera model—whether an ideal pinhole or a strongly distorted lens—is simply not a symmetric 2D Gaussian. Put differently, any framework that leans on linear projective geometry to approximate inherently nonlinear transformations will unavoidably incur approximation errors that no amount of tuning can fully remove. By contrast, our approach dispenses with such projective approximations altogether, aggregating Gaussian density directly along rays within a volumetric rendering formulation that is agnostic to the underlying camera.

### 2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert [2022] introduce the assumption that a ray intersects the Gaussian density function at the point of maximum intensity response. Building on this idea, 3DGRT [Moënné-Loccoz et al. 2024] adopts the heuristic intersection to build a 3D Gaussian ray-tracing framework, enabling ray-space sampling without explicit projection and thereby handling wide-FoV distortions that trouble splatting. The generality of this ray-space formulation, however, comes at a cost: 3DGRT incurs substantial overhead because ray-particle association leans on costly scene-level spatial partitioning structures (BVHs), which ultimately caps its frame rate.

Alternatively, EVER [Mai et al. 2025] replaces the Gaussian representation with simplified constant-density ellipsoids, which admits a closed-form solution for transmittance-weighted ray integration. This substitution, however, narrows the expressive capacity of the representation relative to full 3D Gaussians and, because it too relies on BVH-based structures, remains limited in runtime efficiency. 2DGS [Huang et al. 2024a] instead substitutes 3D Gaussians with 2D surfels, allowing exact ray-surface intersections. HTGS [Hahlbohm et al. 2025] reinstates the full Gaussian expressions while borrowing 2DGS’s ray-splat association. Because that association still lives in screen space, though, it cannot guarantee projective exactness as the field of view widens. Our contribution, in contrast, is to derive closed-form bounds and a complete rendering pipeline directly in a camera-agnostic angular domain (see details in Sec. D); we discuss in detail how our parametrization and

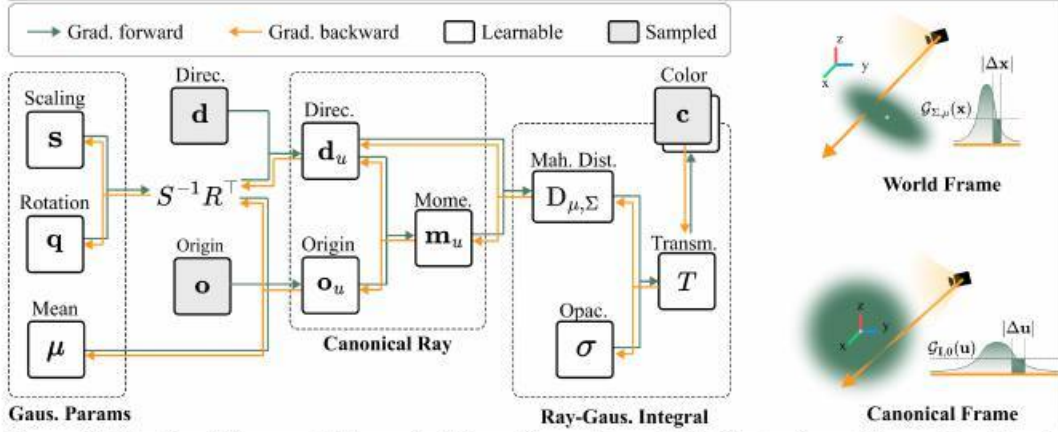


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Illustration of forward and backward gradient propagation from the Gaussian parameters  $s$ ,  $q$ ,  $\mu$ , and the opacity coefficient  $\sigma$  to the transmittance. (Right) Integral of 3D Gaussian density along a ray, where the green box highlights the product of density and ray segment length (see Eq. B.5).

rendering relate to, and differ computationally from, the prior 2DGS and HTGS formulations in Sec. D.3

3DGUT (Wu et al. 2025), a follow-up to 3DGRT, improves runtime efficiency by switching to splatting-based particle association, reaching frame rates on par with 3DGS. Its acceleration, however, hinges on the Unscented Transform (UT), a sampling-based approximation for estimating 2D conics in projective image space. GOF (Yu et al. 2024b) shares a similar limitation, applying the EWA trick directly to fit 2D bounding conics and projective centers for association. In particular, as the field of view grows extreme, imprecise AABB estimation and the resulting inaccurate tile-to-AABB associations surface as visible grid-line artifacts (see Fig. 1 Left).

Departing from all of these, our approach solves the ray-particle association problem directly at the frustum level in 3D space (see Fig. 1 Right), preserving the geometric exactness of ray-based rendering across essentially arbitrary camera models while sustaining rendering speeds comparable to 3DGS. Tab. 1 summarizes representative methods that target projective approximation errors, contrasting them along the several dimensions most relevant to broad, camera-agnostic deployment.

### 3 METHOD

We propose 3DGEER, a Gaussian rendering framework that delivers both geometric exactness and real-time efficiency across cameras of essentially arbitrary FoV. Our method resolves the two bottlenecks that have historically forced a trade-off in prior work—*projection approximation* in splatting-based rendering and *BVH overhead* in ray-based rendering—through three complementary components that are designed to hold uniformly over camera models:

- Ray-based rendering formulation (Sec. 3.1): a closed-form differentiable solution for integrating Gaussian density along 3D rays, eliminating the linearization error inherent to projection-based splatting.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a ray-particle association scheme that replaces BVH traversal with exact frustum-particle tests, yielding accuracy and efficiency together.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that unifies FoV camera models of widely differing geometry under one parameterization, accelerating PBF computation and improving reconstruction quality.

The Appendix further provides: (i) the necessary preliminaries and notations (Sec. B.1), (ii) the complete proof to Sec. 3.1 establishing the connection between maximum response and projective exactness (Sec. B), (iii) the full derivations of backward gradients (Sec. C) and its numerical stability, and (iv) the full derivations of geometrically exact association (Sec. D).

### 3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

A central challenge in geometrically exact Gaussian rendering is obtaining a closed-form expression for integrating density and color along a ray that remains valid regardless of the imaging model. Our strategy is to map each anisotropic Gaussian in world space into a canonical coordinate system in which all Gaussians become isotropic. This reduces the transmittance integral to a measure-preserving change of variables and admits a simple closed-form solution that carries over to any camera whose rays we can parameterize.

**Canonical transformation.** For each Gaussian with mean  $\mu$ , rotation  $R$ , and scaling  $S$ , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which maps a canonical coordinate  $\mathbf{u}$  to world coordinates  $\mathbf{x}$ . This implies  $\Delta\mathbf{x} = RS\Delta\mathbf{u}$  and absorbs the Jacobian determinant into the density measure (see Fig. 2 Right). Consequently, each Gaussian reduces to the shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

**Closed-form transmittance.** For a ray  $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ , its canonical form is  $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$  with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

**Discussion.** This result shows that the transmittance depends only on the ray’s distance to the Gaussian in the canonical space, giving an efficient and geometrically exact closed-form solution that is independent of how the ray was generated. Interestingly, the same expression coincides with the “maximum response” heuristic used in prior works (Keselman & Hebert, 2022; Moëgne-Loccoz et al., 2024); our derivation, however, exposes the projective exactness that underlies it (see details in Sec. B). Moreover, by following our closed-form derivation and the corresponding backward gradient chain (see Fig. 2 Left and details in Sec. C), we sidestep the numerical instabilities that beset prior ray-based work (Yu et al., 2024b) and remove the need for redundant post-processing such as filtering out degenerate Gaussians.

### 3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For any ray-based Gaussian rendering method, both accuracy—especially as camera FoV grows—and runtime efficiency hinge on the ray-particle association strategy, an aspect that prior works have largely left underexplored. To decide which Gaussians contribute to each ray, we simplify the association problem by mapping Camera Sub-Frustums (CSFs) to Particle Bounding Frustums (PBFs)—the tight frustums that enclose each Gaussian at a chosen distance from its center (Fig. 3). A Gaussian is assigned to a CSF whenever its PBF intersects that CSF, and every Gaussian associated with a CSF is then treated as a contributor to all rays within it.

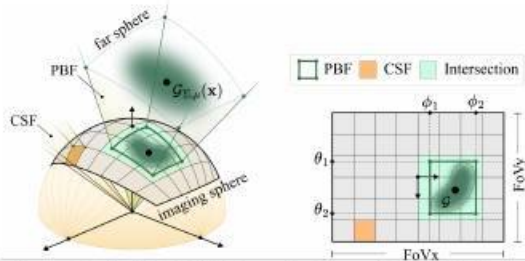


Figure 3: **PBF-CSF Association.** (Left) PBF defined by four planes tangent to a 3D Gaussian ellipsoid. (Right) The intersection between the PBF and CSF, unfolded onto the BEAP imaging plane.

Because this construction is posed in 3D rather than in any fixed image plane, it applies unchanged across camera models.

To efficiently compute a PBF, we first define two spherical angles (see Fig. E.1 in the camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where  $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$  denotes the normalized direction vector pointing to the camera (view) space point  $\mathbf{d}_c$ . The angles  $\theta$  and  $\phi$  correspond to the ray’s incidence angles in the horizontal ( $xz$ ) and vertical ( $yz$ ) planes, respectively, each ranging from  $-\frac{\pi}{2}$  to  $\frac{\pi}{2}$ . We define a PBF by four planes — two aligned with the  $\theta$  angle and two with the  $\phi$  angle (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where  $\theta_1 < \theta < \theta_2$  and  $\phi_1 < \phi < \phi_2$  specify the angular limits.

Given a camera with extrinsic matrix  $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$  and a canonical-to-world transformation matrix  $H$  (See Eq. B.4), we apply the inverse transpose of the camera-to-canonical matrix  $(VH)^{-1}$  to parameterize the corresponding canonical planes using the angular limits defined in view space. This transformation enables us to effectively derive the constraints for the PBFs. Take the bounds of  $\theta$  as an example, the canonical plane is given by:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2} (VH)_2, \quad (8)$$

where  $(VH)_i$  denotes the  $i$ -th row of the matrix, expressed as a column vector.

Assuming a Gaussian ellipsoid defined by a  $\lambda$ -standard-deviation contour, we compute the angular bounds of the local frustum by solving the following quadratic equations, which enforce that the transformed planes are tangent to the ellipsoidal contour:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where  $\circ$  denotes the Hadamard (element-wise) product, and  $\lambda$  is a parameter that controls the spread of the contour, typically set to 3. An identical constraint is applied to  $\mathbf{g}_v^\top$  in order to compute the corresponding angular bounds  $\phi_{1,2}$ . The roots of these quadratic equations determine the tangent values of the angular bounds  $[\theta_1, \theta_2]$  and  $[\phi_1, \phi_2]$ , which define the PBF angular limits. Notably, solving these quadratic equations under canonical space constraints respects the true 3D covariance  $\Sigma_c$  and 3D mean  $\mu_c$  of the Gaussian in the camera space. Specifically, for the bounds of  $\theta$ , we have  $c = \tan \theta_{1,2}$  satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation is efficient to solve (see details in Sec. D.1) and removes the need for the intermediate conic approximations used in EWA or UT, as well as the screen-space constraints of HTGS and 2DGS that cap their usable FoV—so the same association procedure remains valid from narrow-view pinhole to strongly distorted wide-FoV cameras.

### 3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

We introduce the BEAP image representation for several complementary reasons. First, it represents images from large-FoV cameras faithfully and without incurring FoV loss, so a single representation covers the range from perspective to wide-angle capture. Second, it lets image tiles and their corresponding CSFs share an identical parameterization in terms of two  $\theta$  and two  $\phi$  angles, enabling highly efficient PBF association and near-optimal GPU parallelization. Third, the resulting ray sampling is more uniformly distributed in 3D than conventional projective rendering, which empirically improves training efficiency and reconstruction quality across the scenes we evaluate.

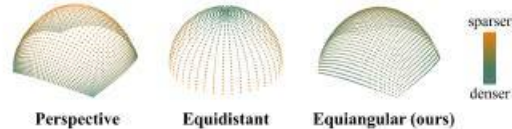


Figure 4: **Comparison of Ray Distributions under Varying Projections.** We project pixels from three imaging spaces onto the unit sphere to compare their ray distributions. Across these projections, our BEAP representation attains the most uniform coverage.

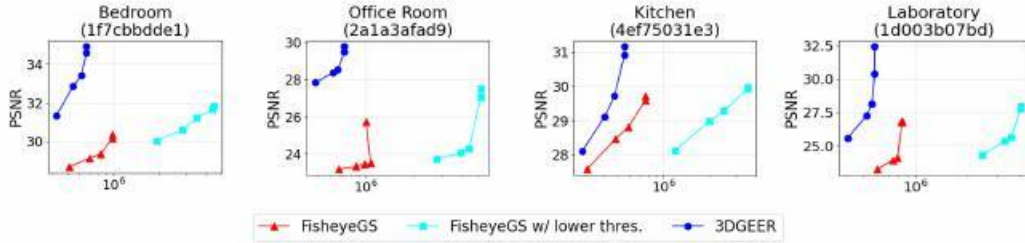


Figure 5: **PSNR $\uparrow$  Trends When Scaling-Up Gaussian Counts.** Across scenes, brute-force scaling fails to close the projective-exactness gap (see full metrics in Fig. K.3 and Tab. K.3).

Specifically, we evenly sample the rays using the spherical angular coordinates  $(\theta, \phi)$  (See Eq. 6), and then map the rays back to a discrete  $w \times h$  image space using a linear transformation consisting of a scaling factor and a center shift as:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where  $\text{FoV}_x, \text{FoV}_y$  indicate the camera’s horizontal and vertical field of view. As illustrated in Fig. 4, sampling rays uniformly in  $(\theta, \phi)$  yields more balanced spatial coverage throughout the view frustum. This stands in contrast to projective sampling, which disproportionately concentrates samples toward peripheral regions, and to equidistant projection, which oversamples near the image center—imbalances that our representation avoids regardless of the camera’s FoV.

## 4 EXPERIMENTS

In this section, we evaluate our proposed method, 3DGEER, across a broad set of datasets that together span pinhole and fisheye camera models, indoor and outdoor scenes, and narrow through wide fields of view. All experiments use an RTX 4090 GPU and 64GB of RAM as the default configuration, unless otherwise stated. Full implementation details are provided in Sec. G.

**Datasets.** To probe generalization across camera types, we evaluate on generic distorted cameras—where projective errors are most consequential—using all scenes from Fisheye ZipNeRF (Barron et al. 2023), five from ScanNet++ (Yeshwanth et al. 2023), and two from Aria (Lv et al. 2025). These span a range of distortion levels: ZipNeRF/ScanNet++ cameras have a  $180^\circ$  diagonal FoV, while Aria has a  $110^\circ$  circular FoV. For the pinhole regime, we use Pinhole ZipNeRF and the widely adopted MipNeRF360 (Barron et al. 2021), so that the same method is stressed across both narrow- and wide-view conditions.

**Evaluation Metrics and Protocol.** We report PSNR $\uparrow$ , SSIM $\uparrow$ , LPIPS $\downarrow$ , runtime performance (FPS $\uparrow$ ), and Gaussian counts $\downarrow$ . To keep comparisons fair across methods with differing internal representations, all results are projected back to the image space for full-FoV evaluation after 30k iterations.

### 4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

(Celarek et al. 2025) argues that simply scaling the number of Gaussians (up to 1M) is enough to close the linear approximation gap between splat-based and ray-based methods. That claim, however, has so far been supported only on small object-centric scenes and leaves open how it behaves once wide-FoV distortions enter the picture. To probe this broader regime, we scale the splat-based fisheye method (FisheyeGS) on ScanNet++ scenes by lowering its densification threshold, producing 3.6–5M Gaussians (H200).

As shown in Fig. 5 the red curve (FisheyeGS) and blue curve (3DGEER) exhibit a clear PSNR gap at matched Gaussian counts, signaling a persistent projective-exactness gap (from 4k to 30k iterations, after which densification typically saturates). The light-blue curve traces the scaled-up FisheyeGS: despite a substantial increase in memory and computation, its performance saturates at



Figure 6: **Extreme-FoV Generalization.** Even at fields of view far beyond those seen in training, 3DGEER reconstructs complete scenes with fewer Gaussians, whereas FisheyeGS suffers severe quality degradation.

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained on either the full FoV or only the central region, then evaluated on the full, central, and peripheral regions to probe robustness across the range of distortion within a frame. Region-wise metrics are weighted by pixel masks; LPIPS is omitted as it is not directly separable by region (Tab. K.2 shows full stats).

| Method        | Training FoV | Training Space | Full FoV        |                 |                    | Central         |                 | Peripheral      |                 |
|---------------|--------------|----------------|-----------------|-----------------|--------------------|-----------------|-----------------|-----------------|-----------------|
|               |              |                | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ |
| 3DGS          | Central      | Persp.         | -               | -               | -                  | 31.26           | 0.948           | -               | -               |
| FisheyeGS     | Full         | Eq.            | 27.81           | 0.946           | 0.139              | 32.44           | <b>0.956</b>    | 23.28           | 0.914           |
| EVER          | Full         | KB             | 29.47           | 0.924           | 0.167              | 29.93           | 0.924           | 28.72           | 0.925           |
| 3DGUT         | Full         | Eq.            | 30.64           | 0.944           | 0.150              | 31.87           | 0.945           | 28.84           | 0.937           |
| 3DGEER (Ours) | Central      | BEAP           | 29.93           | 0.949           | 0.130              | <b>32.86</b>    | 0.954           | 26.64           | 0.930           |
| 3DGEER (Ours) | Full         | BEAP           | <b>31.50</b>    | <b>0.953</b>    | <b>0.126</b>       | 32.64           | 0.955           | <b>28.94</b>    | <b>0.945</b>    |

29.3 PSNR, whereas ours reaches 32.1 with only 550–700K Gaussians. This pattern indicates that the gap is structural rather than a matter of capacity, and thus is not something scaling alone can overcome in the wide-FoV setting. Full statistics are reported in Tab. K.3

#### 4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We begin by evaluating 3DGEER against representative baselines across both fisheye and pinhole benchmarks, adding analysis tailored to each dataset so that the comparison covers the full breadth of conditions.

**ScanNet++.** As shown in Tab. 2, 3DGEER consistently outperforms all baselines across metrics. To disentangle distortion effects, we evaluate central and peripheral regions separately (details in Sec. G). Competing methods hold up reasonably at the center but degrade toward the periphery, whereas 3DGEER sustains high fidelity throughout the frame, producing sharper boundaries and fewer artifacts (Fig. K.4). Notably, even when trained only on central crops, 3DGEER still surpasses FisheyeGS across all regions—evidence that its advantage does not depend on having seen the peripheral, high-distortion content during training.

Qualitatively (Fig. 6), we further probe extreme-FoV generalization by training on the original dataset FoV (180° diagonal FoV) and testing at substantially wider FoVs (180° FoV, circular shape). Even under this train–test mismatch, 3DGEER reconstructs complete scenes with fewer Gaussians, while splatting-based methods such as FisheyeGS degrade markedly at the periphery (see additional extreme-FoV visuals in Fig. K.1–K.2).

**ZipNeRF.** Because the dataset provides frame-wise aligned pinhole and fisheye views, it is well suited to testing transfer between camera models. Beyond standard fisheye evaluation, we therefore run *cross-camera generalization*—training on fisheye

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Alongside quality metrics, FPS is reported on an RTX 4090 as the default benchmark. Numbers taken from the original works are marked with  $\dagger$  (measured on RTX 6000 Ada) and  $\ddagger$  (measured on RTX 5090).

| Method | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | FPS $\uparrow$ |
|--------|-----------------|-----------------|--------------------|----------------|
| 3DGS   | 27.21           | 0.815           | 0.214              | <b>343</b>     |
| EVER   | 27.51           | <b>0.825</b>    | 0.233              | 36             |

Table 3: **Comparison on the ZipNeRF Dataset.** The experiments follow a cross-camera generalization protocol: each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and then evaluated separately on both Fisheye and Pinhole test sets, directly measuring transfer across camera models (Tab. K.9 shows full stats).

| Train Data    | 1/8 FE          |              | 1/8 FE          |              | 1/8 FE             |              | 1/4 PH          |              | 1/4 PH          |              | 1/4 PH             |              |
|---------------|-----------------|--------------|-----------------|--------------|--------------------|--------------|-----------------|--------------|-----------------|--------------|--------------------|--------------|
| Test Data     | FE              | PH           | FE              | PH           | FE                 | PH           | FE              | PH           | FE              | PH           | FE                 | PH           |
| Method        | PSNR $\uparrow$ |              | SSIM $\uparrow$ |              | LPIPS $\downarrow$ |              | PSNR $\uparrow$ |              | SSIM $\uparrow$ |              | LPIPS $\downarrow$ |              |
| FisheyeGS     | 23.18           | 26.44        | 0.858           | 0.868        | 0.211              | 0.239        | 19.43           | 26.61        | 0.791           | <b>0.889</b> | 0.247              | <b>0.212</b> |
| EVER          | 25.17           | 26.14        | 0.880           | 0.851        | 0.153              | 0.237        | 22.90           | 26.45        | 0.835           | 0.862        | <b>0.207</b>       | 0.222        |
| 3DGUT         | 24.77           | 25.59        | 0.879           | 0.804        | 0.183              | 0.324        | 18.61           | 25.96        | 0.662           | 0.841        | 0.300              | 0.270        |
| 3DGEER (Ours) | <b>26.24</b>    | <b>27.62</b> | <b>0.897</b>    | <b>0.888</b> | <b>0.140</b>       | <b>0.214</b> | <b>23.39</b>    | <b>27.61</b> | <b>0.846</b>    | 0.863        | 0.209              | 0.254        |

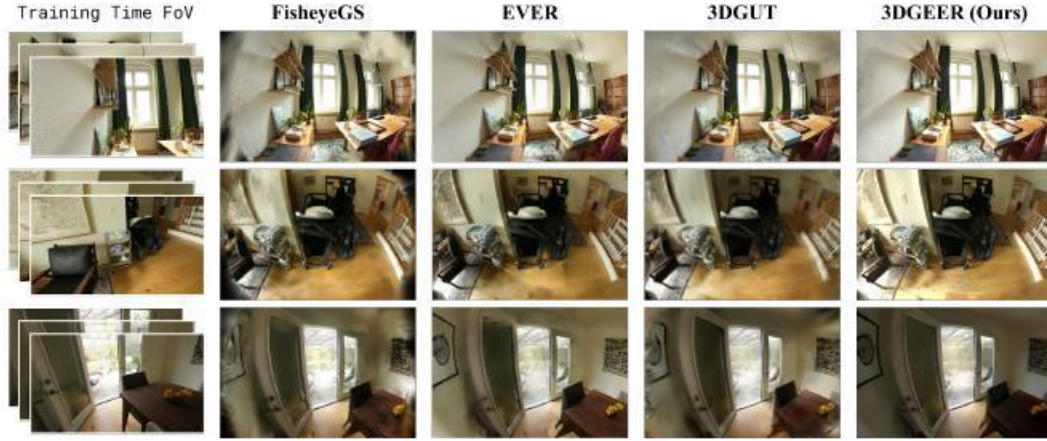


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** The gap widens in the most demanding Pinhole (PH) train – Fisheye (FE) test setup, precisely the cross-camera transfer case where our method holds a clear advantage.

and testing on pinhole, and vice versa—to gauge robustness across imaging setups. As shown in Tab. 3, 3DGEER consistently outperforms all methods across metrics, fully exploiting large-FoV training data through geometrically exact rendering. Even when trained on pinhole data, 3DGEER still delivers notable PSNR gains on both views. FisheyeGS, by contrast, performs well only in the pinhole-to-pinhole case, and 3DGUT drops sharply when tested on fisheye after pinhole training. Broadly, EVER and 3DGEER generalize best thanks to their geometric exactness, with 3DGEER superior in every case. Qualitatively (Fig. 7), 3DGEER recovers finer details and resists peripheral artifacts more strongly in the most demanding pinhole-to-fisheye setting.

To confirm that these gains are not specific to wide-FoV capture, we further evaluate 3DGEER on the **MipNeRF360** dataset under standard narrow-FoV conditions. As shown in Tab. 4, 3DGEER sets a new state of the art across all metrics. It runs over  $5\times$  faster than exact ray-marching/tracing methods such as EVER and 3DGRT, and is the only projective-exact method whose runtime is comparable to 3DGS. It also consistently surpasses the projective-approximation baselines (3DGS, 3DGUT) in PSNR, SSIM, and LPIPS. That the same method leads in this narrow-FoV regime, having already led under wide-FoV distortion, points to broad applicability rather than a niche specialization. Additional quantitative results on **Aria**, an egocentric fisheye dataset (moderate  $110^\circ$  FoV, circular shape) with low-light imaging, along with further visuals, are provided in Sec. J.

Table 5: **Training-Time Transmittance Replacement on ScanNet++**. Substituting splatting-based transmittance inflates the Gaussian count owing to its larger approximation error, yet a perceptual gap persists relative to our projective-exact formulation (Tab. K.7 shows the full stats).

| Transmittance Method    | Full FoV     |              |              | Central      |              | Peripheral   |              | Avg. Gaussian Number (k) ↓ |
|-------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|----------------------------|
|                         | PSNR ↑       | SSIM ↑       | LPIPS ↓      | PSNR ↑       | SSIM ↑       | PSNR ↑       | SSIM ↑       |                            |
| 3DGS Splats             | 22.86        | 0.878        | 0.177        | 27.16        | 0.884        | 18.56        | 0.855        | 1396.4                     |
| FisheyeGS Splats        | 27.90        | 0.948        | 0.141        | 32.49        | 0.957        | 23.36        | 0.917        | 920.6                      |
| Projective-Exact (Ours) | <b>31.50</b> | <b>0.953</b> | <b>0.126</b> | <b>32.64</b> | <b>0.955</b> | <b>28.94</b> | <b>0.945</b> | <b>591.8</b>               |

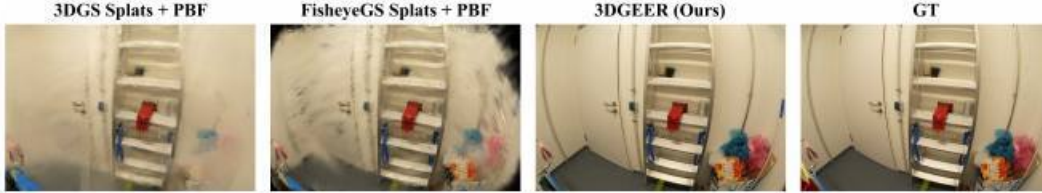


Figure 8: **Rendering-Time Transmittance Func. Replacement**. Swapping projective-exact transmittance for the splatting approximation at test time alone already leads to mismatched culling.

#### 4.3 COMPREHENSIVE RUNTIME ANALYSIS

We next analyze runtime on RTX 4090, comparing 3DGEER with splatting-based methods across two regimes (3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**). Although our ray-based rendering costs slightly more in the rendering stage, PBF’s efficient ray-particle association sharply cuts Gaussian duplication and sorting overhead (see detailed runtime in Tab. K.4), so the overall efficiency stays comparable to optimized splatting pipelines in both the pinhole and fisheye cases.

Concretely, PBF produces very tight bounds, associating each tile with only  $\sim 475$  Gaussians on average—3–5 $\times$  fewer than EWA or UT (Tab. K.6). This directly accounts for the 2.5–5 $\times$  runtime speedup at the association stage (Tab. K.5). Projection-based schemes (EWA, UT), by contrast, conservatively overestimate screen-space extent, causing excessive overdraw and inefficient memory access. Even accelerated with SnugBox (Hanson et al., 2025), they remain hampered by miscentered density approximations and often amplify artifacts (Fig. 1). Beyond speed, PBF preserves geometric accuracy and numerical stability, removing projective approximation errors entirely. As Tab. K.6 shows, even when ray-based rendering is applied to Gaussian fields trained under other association schemes, PBF stays artifact-free—indicating that its benefits transfer across training pipelines, not only our own.

#### 4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the effect of our projective-exact transmittance formulation (Eq. 4), we swap it for the widely used local affine splats (i.e., the 2D conic approximation adopted in 3DGS and FisheyeGS).

**Rendering-Time Approx.** To sharpen the contrast between the two rendering formulations, we replace the transmittance with the splatting-based approximation at test time only, leaving every projective-exact component intact during training. As shown in Fig. 8 once the transmittance relies on the splat approximation while associations are still computed with our exact projective geometry, the resulting inconsistency produces mismatched culling. This experiment magnifies and explains the gridline artifact reported in 3DGUT (Fig. 1), where the same kind of mismatch arises in the opposite direction—ray-based transmittance paired with conic-approximate association.

**Training-Time Approx.** As reported in Tab. 5 on **ScanNet++**, when splatting-based transmittance is also used during training while association stays projective-exact, the optimizer and densification partially absorb the approximation error and the cross-view inconsistency that splatting introduces. The larger loss drives stronger gradients throughout training, so the model adds a substantially larger (1.6–2.4 $\times$ ) number of Gaussians to fit the scene. Yet even with this inflated count, the perceptual

Table 6: **BEAP vs. Alternative Supervision Schemes.** Across every metric on ScanNet++, BEAP consistently outperforms the alternative projections. (Tab. K.8 shows full stats).

| Method          | Training<br>FoV | Training<br>Space | Full FoV        |                 |                    | Central         |                 | Peripheral      |                 |
|-----------------|-----------------|-------------------|-----------------|-----------------|--------------------|-----------------|-----------------|-----------------|-----------------|
|                 |                 |                   | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ |
| 3DGEER w/o BEAP | Central         | Persp.            | 29.84           | 0.943           | 0.131              | 32.21           | 0.951           | 26.23           | 0.915           |
| 3DGEER w/o BEAP | Full            | Persp.            | 21.11           | 0.853           | 0.300              | 21.32           | 0.855           | 20.46           | 0.846           |
| 3DGEER w/o BEAP | Full            | Eq.               | 31.05           | 0.948           | 0.135              | 32.21           | 0.952           | 28.56           | 0.935           |
| 3DGEER (Ours)   | Full            | BEAP              | <b>31.50</b>    | <b>0.953</b>    | <b>0.126</b>       | <b>32.64</b>    | <b>0.955</b>    | <b>28.94</b>    | <b>0.945</b>    |

gap the approximation opens up remains, showing that the error cannot be corrected through densification alone.

#### 4.5 BEAP ON/OFF STUDY

We further compare BEAP supervision against other widely used projection models on **ScanNet++**. Here we hold the projective-exact integral and PBF fixed for Gaussian transmittance and association, and vary only the ray-sampling distribution used for supervision (pinhole, equidistant, and BEAP). As shown in Tab. 6 and Fig. K.5 BEAP-based supervision outperforms every alternative across all metrics. Its more uniform sampling yields smoother, more consistent gradients that converge better on fine details.

The discrepancy widens under pinhole cameras. At a fixed resolution, a well-balanced information distribution can matter more than simply widening the FoV. Notably, training in the perspective domain over the full FoV can even underperform the central region (see Tab. 6 rows 1–2): large-FoV images (Fig. K.2 Left) sample angles highly non-uniformly, concentrating informative content at the center while the less informative periphery occupies most of the image area and is disproportionately oversampled. This suggests the benefit of balanced angular sampling is a general one, applying whenever the camera’s angular resolution varies across the frame rather than being tied to any single projection.

## 5 CONCLUSIONS

We presented 3DGEER, a complete, closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles and designed to apply uniformly across camera models. Complementing this formulation, we introduced an efficient PBF method for ray-particle association that enables high-speed rendering while preserving exactness in projective geometry, together with the BEAP image representation for effective color supervision under a wide range of FoV camera models. Extensive experiments—spanning pinhole and fisheye datasets, indoor and outdoor scenes, and narrow through extreme fields of view—show that 3DGEER consistently outperforms state-of-the-art methods, with especially strong results in the challenging wide-FoV settings where prior methods falter. Taken together, these results position 3DGEER as a general-purpose rendering framework rather than a solution tailored to any single camera or scene type.

## ETHICS STATEMENT

This work builds on publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and involves no human subjects, private data, or personally identifiable information. The methods we develop are aimed at advancing 3D scene reconstruction and rendering research broadly, with natural applications spanning XR, robotics, autonomous driving, and other settings that rely on wide-FoV or generically distorted cameras. We foresee no direct harmful consequences, but note that any misuse of high-fidelity 3D reconstruction could raise privacy concerns. To mitigate this, we confine our experiments to research datasets in common use across the vision community.

## REPRODUCIBILITY STATEMENT

We have made substantial efforts to ensure reproducibility across the full range of settings we report. All mathematical derivations and proofs appear in the Appendix (Secs. B, C and D). Every dataset used in our experiments is publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF), together covering both pinhole and fisheye cameras. Preprocessing, training protocols, and evaluation metrics are detailed in Sec. G and our implementation will be released with training and evaluation scripts upon publication to further support reproduction.

## REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

- 
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

- 
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

---

## CONTENTS

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                                   | <b>1</b>  |
| <b>2 Related Works</b>                                                  | <b>3</b>  |
| 2.1 Splatting-Based Gaussian Rendering                                  | 3         |
| 2.2 Ray-Based Gaussian Rendering                                        | 3         |
| <b>3 Method</b>                                                         | <b>4</b>  |
| 3.1 Rendering with Exact Projective Geometry                            | 5         |
| 3.2 Ray-Particle Association with Exact Projective Geometry             | 5         |
| 3.3 Bipolar Equiangular Projection (BEAP)                               | 6         |
| <b>4 Experiments</b>                                                    | <b>7</b>  |
| 4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras      | 7         |
| 4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis            | 8         |
| 4.3 Comprehensive Runtime Analysis                                      | 10        |
| 4.4 Local Affine Approximation vs. Projective-Exact Integral            | 10        |
| 4.5 BEAP On/Off Study                                                   | 11        |
| <b>5 Conclusions</b>                                                    | <b>11</b> |
| <b>a Ethics Statement</b>                                               | <b>12</b> |
| <b>b Reproducibility Statement</b>                                      | <b>12</b> |
| <b>c LLM Usage Statement</b>                                            | <b>18</b> |
| <b>A Appendix: Exactness and Limitation</b>                             | <b>18</b> |
| <b>B Appendix: Ray-Gaussian Transmittance with Projective Exactness</b> | <b>18</b> |
| B.1 Preliminaries And Notations                                         | 18        |
| B.2 Differentiable Rendering with Projective Exactness                  | 19        |
| B.3 Ray Integral of Isotropic 3D Gaussian                               | 20        |
| <b>C Appendix: Cookbook of Gradient Computation</b>                     | <b>21</b> |
| C.1 Chain Rule                                                          | 21        |
| C.2 Two-Stage Propagation                                               | 21        |
| C.3 Gradients for Canonical Ray                                         | 21        |
| C.4 Gradients for Whitening Matrix                                      | 22        |
| C.5 Gradients for Scaling and Rotation                                  | 22        |
| C.6 Gradients for 3D Mean                                               | 22        |
| C.7 Numerical Stability of Our Differentiable Framework                 | 23        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>D Appendix: Additional Details in Association</b>        | <b>23</b> |
| D.1 Particle Bounding Frustum (PBF) Solver                  | 24        |
| D.2 Details in Computing CSF-PBF Intersection               | 24        |
| D.3 Relation to Prior Works                                 | 25        |
| <b>E Appendix: BEAP Image Inverse Interpolation</b>         | <b>26</b> |
| Pinhole Camera Model                                        | 26        |
| KB Fisheye Model                                            | 27        |
| <b>F Appendix: High-level Algorithmic Summary</b>           | <b>27</b> |
| <b>G Appendix: Additional Implementation Details</b>        | <b>27</b> |
| Baselines                                                   | 27        |
| Implementation Details                                      | 27        |
| Central and peripheral regions in ScanNet++ evaluation      | 28        |
| Cross-camera generalization evaluation on ZipNeRF           | 28        |
| <b>H Appendix: EWA Splatting Preliminaries</b>              | <b>28</b> |
| From camera coordinates to ray coordinates                  | 28        |
| Local affine approximation                                  | 29        |
| Splatting approximation error                               | 29        |
| <b>I Appendix: Advantage over Prior Ray-Tracing Methods</b> | <b>29</b> |
| <b>J Appendix: Results on Egocentric Dataset</b>            | <b>30</b> |
| <b>K Appendix: Additional Material and Results</b>          | <b>32</b> |
| K.1 Additional Material.                                    | 32        |
| K.2 Additional Results.                                     | 32        |

## LLM USAGE STATEMENT

We used a large language model (ChatGPT, OpenAI) as a writing assistant. Specifically, the LLM was employed to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic edits to figure captions and section transitions, and (iii) provide alternative phrasings for technical descriptions without altering the scientific content.

The LLM was not involved in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors. The authors take full responsibility for the content of this paper.

## A APPENDIX: EXACTNESS AND LIMITATION

We clarify that our notion of exactness refers to *eliminating linear approximation in projective geometry* during both the “rendering” and “ray-particle association” stages, a property that holds across camera models and under which the residual error does not shrink as the number of Gaussians grows (see Sec. 4.1). We do not claim exactness in the full physical volumetric sense: our framework still involves approximations such as ordering, self-attenuation, and overlapping, which we treat as its principal limitations.

**Limitations.** Our formulation integrates each Gaussian independently; it therefore does not explicitly resolve self-occlusion, and ordering is handled through depth-based sorting. As the supplementary videos show, this can occasionally produce popping artifacts. To our knowledge, EVER (Mai et al. 2025) is the only method that analytically resolves ordering and self-occlusion, which it does by approximating each Gaussian as a constant-density ellipsoid—at a substantial cost in rendering speed.

Importantly, prior ray-tracing-based Gaussian renderers such as Fuzzy Metaballs (Keselman & Hebert 2022) and subsequent methods including 3DGR/UT (Moënné-Loccoz et al. 2024; Wu et al. 2025) all rely on the “maximum-response” assumption for ray-Gaussian interaction. This assumption inherently sets aside ordering and self-attenuation, yet has been shown to introduce negligible error in practice, independent of the number of Gaussians (see Celarek et al. 2025 Fig. 9-11 / Sec. 7.1). It follows that our depth-based ordering attains the same practical accuracy as existing ray-tracing-based Gaussian renderers.

**Future work.** Although the practical impact of ordering and self-occlusion stays small in current Gaussian-based ray tracing, these factors ultimately bound exact volumetric correctness. One promising direction is to incorporate analytic or learnable self-attenuation models—perhaps inspired by ellipsoidal integration (e.g., EVER Mai et al. 2025)—without incurring heavy runtime costs. Another is to handle ordering more principledly, which could further suppress the rare popping artifacts seen under extreme configurations. These limitations are largely orthogonal to our contributions on projective exactness, and resolving them would extend the framework’s robustness to an even broader range of large-scale and high-dynamic scenes.

## B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

### B.1 PRELIMINARIES AND NOTATIONS

We adopt the classic parameterization of 3D Gaussian primitives introduced in prior work (Zwicker et al. 2001), which is standard across the splatting and ray-based literature alike. Each 3D Gaussian distribution is defined by a mean position  $\mu \in \mathbb{R}^3$  and a 3D covariance matrix  $\Sigma$  in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left( -\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where we set the normalization constant  $\rho$  to  $\sqrt{2\pi}$ , and  $\mathbf{x} \in \mathbb{R}^3$  denotes a point in world space. This formulation departs from recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al.

2025], which drop the determinant term  $|\Sigma|^{1/2}$  to simplify splatting or association during rendering; retaining it lets our subsequent derivation stay exact for any camera. The covariance matrix takes the form  $\Sigma = RSS^\top R^\top$ , given a scaling matrix  $S = \text{diag}(\mathbf{s})$  constructed from the scaling vector  $\mathbf{s} \in \mathbb{R}^3$ , and a rotation matrix  $R \in \text{SO}(3)$  derived from a unit quaternion  $\mathbf{q} \in \mathbb{R}^4$ .

To evaluate the transmittance contributed by a single 3D Gaussian particle along a ray  $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ , we have:

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where  $\sigma$  is the opacity coefficient associated with the Gaussian.

Finally, considering a set of 3D Gaussian particles sorted by depth contributing to the rendering color of the given ray, the rendered color  $C$  is computed as:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where  $c_i(\mathbf{o}, \mu_i)$  gives the view-dependent color of the  $i$ -th Gaussian as seen from the optical center  $\mathbf{o}$ , parameterized with spherical harmonics (SH) (Kerbl et al., 2023). By minimizing the reconstruction loss between the rendered and observed colors along each ray, the parameters  $\mathbf{s}$ ,  $\mathbf{q}$ ,  $\mu$ , the coefficient  $\sigma$ , and the SH coefficients are jointly optimized via backpropagation—a procedure that remains unchanged whatever camera the rays are drawn from.

## B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The core challenge in achieving projective-exact Gaussian rendering within a ray-marching formulation is to obtain a closed-form solution for integrating Gaussian density and color along a 3D ray that does not depend on the imaging model. To this end, we first transform the ray—relative to each Gaussian—into a canonical coordinate system in which the transmittance integral (see Eq. B.2) is preserved and the Gaussians become isotropic. We then derive the exact ray color from first principles, in a form that carries over to arbitrary cameras.

Specifically, we define a canonical coordinate system shared across all 3D Gaussian primitives. A point  $\mathbf{u} \in \mathbb{R}^3$  in this canonical space is mapped to world coordinates for each Gaussian via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

From this transformation, an infinitesimal segment  $\Delta \mathbf{u}$  along a ray in the canonical frame corresponds to a segment  $\Delta \mathbf{x} = RS \Delta \mathbf{u}$  in the world frame.

To ensure that the transmittance contributed by each Gaussian along the ray is preserved across coordinate systems, we require that the elementary transmittance (i.e., the product of density and segment length) remains invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint implies that the Jacobian determinant is absorbed into the measure. Note that  $\left| \frac{\Delta \mathbf{x}}{\Delta \mathbf{u}} \right| = |RS| = |\Sigma|^{1/2}$ . We can thus express the corresponding canonical density as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp \left( -\frac{1}{2} \mathbf{u}^\top \mathbf{u} \right), \quad (\text{B.6})$$

which is isotropic and shared across all primitives in canonical space.

This leads to an important conclusion that the exact closed-form solution for the transmittance integral of each Gaussian along the ray  $\mathbf{r}(t)$  can thus be obtained by computing the accumulated density along the transformed ray  $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$  in the canonical isotropic Gaussian. Note that  $t < 0$  is always negligible unless the optical center is within 3-sigma, then all 3D Gaussian rendering frameworks cull the Gaussian when optimization.

This admits a *closed-form* solution in terms of the perpendicular distance from the ray to the Gaussian center, as captured in Theorem I

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t \mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where  $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$  denotes the perpendicular distance from the Gaussian center (i.e., the origin in the canonical space) to the transformed ray  $\mathbf{r}_u(t)$ . Intuitively, this quantity also represents the minimal *Mahalanobis distance* from any points on the ray to the Gaussian in world space, measuring how close the ray passes by the particle distribution.

The remaining question is just about how to compute the perpendicular distance from the canonical origin to the transformed ray. We derive the transformed ray's direction vector  $\mathbf{d}_u$  and moment vector  $\mathbf{m}_u$  as:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where  $\mathbf{o}_u = (S^{-1}R^\top)(\mathbf{o} - \boldsymbol{\mu})$  denotes the optical center in canonical space. Based on this, the square of  $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$  can be efficiently calculated as:

$$(D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

By substituting this distance square into Eq. B.7 and then the derived transmittance value into Eq. B.3 we compute the accumulated color along the ray.

### B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

**Theorem 1.** *Given a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form proportional to:*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where  $D$  is the perpendicular distance from the Gaussian center to the ray.

*Proof.* We first apply a translation to the coordinate system such that the Gaussian  $\mathcal{G}$  is centered at the origin, and the ray  $\mathbf{r}_u$  is translated accordingly. Next, we rotate the entire scene so that the ray direction is aligned with the  $z$ -axis. Due to the translation-invariance of the integral and the isotropy of the standard Gaussian, these transformations do not affect the result of the integral.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the  $z$ -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ \mathbf{o}_v \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where  $D^2 = \frac{u_0^2 + v_0^2}{2}$  denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Thus, the integral is proportional to  $\exp\left(-\frac{1}{2}D^2\right)$ , as stated in theorem.  $\square$

Note that the density function shared by our 3D primitives in the canonical space (see Eq. B.6) follows the same formulation as in the isotropic case. Given the world-coordinate ray  $(\mathbf{o}, \mathbf{d})$  as the input of  $D$ , its value also depends on the world-space Gaussian parameters  $\mu, \Sigma$ , which are used to transform the ray into the canonical coordinate system. Therefore, for each Gaussian contributing to the ray, we derive the transmittance as:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 when picking  $\rho = \sqrt{2\pi}$ .

## C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

### C.1 CHAIN RULE

Recall that  $T = \sigma\alpha$  is the ray-Gaussian transmittance (Eq. B.7), where  $\sigma$  is the opacity coefficient, and  $\alpha$  is the ray-Gaussian integral. We denote  $\kappa = D_{\mu, \Sigma}^2$  as the minimal squared Mahalanobis distance from the ray to the Gaussian distribution,  $S$  as the Gaussian scaling, and  $R$  as the rotation matrix in the world space. Additionally, given  $\Sigma = RSS^\top R^\top$  as the 3D covariance matrix of the Gaussian in the world space, we have  $W_{\text{pca}}$  as the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad (\text{C.1})$$

where  $S^{-1} = \text{diag}\left(\frac{1}{s}\right)$ .

As illustrated in Fig. 2 we can apply the chain rule to find derivatives *w.r.t.* the Gaussian parameters, e.g., scaling vector  $\mathbf{s}$ , rotation quaternion  $\mathbf{q}$ , and mean  $\mu$ :

$$\frac{\partial T}{\partial \mathbf{s}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial \mathbf{q}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (\text{C.4})$$

### C.2 TWO-STAGE PROPAGATION

To reduce overhead in shared propagation, we structure our gradient computation after the two-stage approach of 3DGS (Kerbl et al. 2023), comprising `RenderCUDA` (RC) and `PreprocessCUDA` (PC). The former performs ray-wise accumulation, efficiently accelerated by our Particle Bounding Frustum (PBF) association (see Section 3.2), while the latter operates at Gaussian-wise complexity and exploits view-frustum culling for additional speedup. This decomposition is independent of the camera model, so the same two-stage schedule applies across the imaging conditions we consider.

According to Eq. B.7 we can respectively write the gradient *w.r.t.* opacity coefficient and the shared partial derivative for Gaussian parameters in RC as:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients to the Gaussian parameters can then be back-propagated from the squared distance, e.g.,  $\frac{\partial \kappa}{\partial \mathbf{s}}, \frac{\partial \kappa}{\partial \mathbf{q}}, \frac{\partial \kappa}{\partial \mu}$ .

### C.3 GRADIENTS FOR CANONICAL RAY

We then backpropagate gradients to the canonical ray parameters: the origin  $\mathbf{o}_u$ , direction  $\mathbf{d}_u$ , and moment  $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$ , as defined in Eq. B.8 in our paper. These quantities directly influence the Mahalanobis distance in Eq. B.9 and are critical for capturing canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left( \frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = - \left( \frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

#### C.4 GRADIENTS FOR WHITENING MATRIX

Recall that  $\mathbf{d}_u = \mathbf{W}_{\text{pca}} \mathbf{d}$  and  $\mathbf{o}_u = \mathbf{W}_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$ , where  $\mathbf{o} = -R_c^\top \mathbf{t}_c$  denotes the world space optical center, and  $\mathbf{d}$  denotes the ray direction. Once the gradients *w.r.t.* the ray parameters are obtained, we further propagate them to the PCA whitening matrix  $\mathbf{W}_{\text{pca}}$  via outer products with the world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right| &= \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{RC}} + \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here, the gradient backpropagated through  $\mathbf{d}_u$  is accumulated ray-wise in `RenderCUDA`, while the gradient through  $\mathbf{o}_u$  is computed Gaussian-wise in `PreprocessCUDA`. These two components are then summed to yield the complete gradient *w.r.t.* the PCA transformation matrix  $\mathbf{W}_{\text{pca}}$ .

#### C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients  $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{s}}$ , we have:

$$\left. \frac{\partial \mathbf{W}_{\text{pca}}^{i,j}}{\partial s_k} \right|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

To derive gradients for rotation  $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{q}}$ , given the quaternion form  $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$ , we can write the whitening matrix as:

$$\mathbf{W}_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1}(\frac{1}{2} - q_j^2 - q_k^2) & \frac{1}{s_1}(q_i q_j + q_r q_k) & \frac{1}{s_1}(q_i q_k - q_r q_j) \\ \frac{1}{s_2}(q_i q_j - q_r q_k) & \frac{1}{s_2}(\frac{1}{2} - q_i^2 - q_k^2) & \frac{1}{s_2}(q_j q_k + q_r q_i) \\ \frac{1}{s_3}(q_i q_k + q_r q_j) & \frac{1}{s_3}(q_j q_k - q_r q_i) & \frac{1}{s_3}(\frac{1}{2} - q_i^2 - q_j^2) \end{pmatrix}. \quad (\text{C.11})$$

As a result, we derive the following end gradients for the quaternion  $\mathbf{q}$  as:

$$\begin{aligned} \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_r} \right|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_i} \right|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_1} & \frac{q_k}{s_1} \\ \frac{q_j}{s_2} & -\frac{2q_i}{s_2} & \frac{q_r}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \end{pmatrix}, \\ \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_j} \right|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_j}{s_3} \end{pmatrix}, \quad \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_k} \right|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_i}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

#### C.6 GRADIENTS FOR 3D MEAN

Meanwhile, we compute end gradients *w.r.t.* the Gaussian 3D mean  $\boldsymbol{\mu}$  as:

$$\left. \frac{\partial \kappa}{\partial \boldsymbol{\mu}} \right|_{\text{PC}} = -\mathbf{W}_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) the result when degenerate Gaussians go unfiltered in GOF, and (Right) the stable output obtained once our forward and backward formulations are applied.

### C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choices of forward formulation, backward gradient computation, and intermediate cache variables together have a decisive effect on numerical stability across the diverse Gaussian shapes that arise in practice. Prior approaches such as GOF (Yu et al., 2024b) suffer from instability in backward propagation and therefore lean on additional 3D filtering heuristics to stabilize training. Our closed-form formulation, together with its exact gradient chain, instead avoids such instability altogether and removes the need for any auxiliary filtering—so the same pipeline remains robust without per-case tuning.

The key distinction from our gradient and rendering pipeline lies in that prior methods adopt an alternative expansion of  $\mathbf{m}_u^\top \mathbf{m}_u$ . Ideally, disregarding numerical issues, the following two forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

However, whenever a 3D Gaussian has one axis scaled close to zero—a common situation across real scenes—the canonical-space vectors  $\mathbf{o}_u$  and  $\mathbf{d}_u$  become nearly parallel. In this regime,  $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$ , while both  $\|\mathbf{o}_u\|$  and  $\|\mathbf{d}_u\|$  diverge. The term  $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$  is then highly prone to numerical overflow and, in practice, can even return negative values, which in turn destabilizes backward propagation.

Fig. C.1 (Left) illustrates the numerical artifacts produced by GOF: without additional 3D filtering to remove degenerate Gaussians, severe artifacts emerge. The snowflake noise traces back to numerical overflow in Eq. C.14, where negative values are returned prematurely and extremely small positive values inflate the Gaussian intensity. Because the EWA approximation with its Jacobian and conic stays numerically stable, these artifacts stem chiefly from inaccurate maximum-response computation, which likewise induces gridline patterns.

Our CUDA rasterizer, by contrast, computes both the cross product and its gradients directly (Eq. B.9 (forward) and Eqs. C.6, C.9 (backward)), sidestepping this issue at its root. The forward pass is thus artifact-free and the backward pass yields accurate gradients. Fig. C.1 (Right) shows the effect of applying our forward and backward formulations to GOF: even without any additional 3D filtering to exclude degenerate Gaussians (flat disk-like or needle shapes), our approach removes the artifacts caused by numerical instability—suggesting the formulation is a drop-in stabilizer usable beyond our own pipeline.

## D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 gives our detailed derivation. Section D.2 covers the implementation considerations needed for stable PBF across the full span from large-FoV to omnidirectional cameras. Section D.3 clarifies the derivational relationships and states explicitly the conditions under which prior formulas reduce to special cases of ours.

## D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Recall Eq. 8 we define  $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$  as the plane transformation matrix from the view space to the canonical space. Take the unknown bounds for  $c = \tan \theta_{1,2}$  as example, we have:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where  $P_i$  denotes  $i$ -th row of the matrix, expressed as a  $4 \times 1$  column vector, and  $P_{ij}$  further denotes  $j$ -th element. Recall Eq. 9 assuming we are computing the bounding box of the 1-standard-deviation contour ellipsoid of the Gaussian, *i.e.*,  $\lambda = 1$ , then we have the following quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where  $\mathbf{f} = (1, 1, 1, -1)$  is a  $1 \times 4$  row vector. Here we denote the equation as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where  $\mathcal{T}$  is a  $3 \times 3$  upper triangular matrix defined with  $\mathcal{T}_{ij} = (\mathbf{f} \circ P_i^\top) \cdot P_j$ .

Similarly, for the bounds of  $\phi$ , we have  $c = \tan \phi_{1,2}$  satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Interestingly, given a camera-view configuration  $[R_c \mid \mathbf{t}_c]$ , we find that the parameter matrix  $\mathcal{T}$  can be efficiently computed from the Gaussian's view-space covariance  $\Sigma_c = R_c R S S^\top R^\top R_c^\top$  and 3D mean  $\mu_c = R_c \mu + \mathbf{t}_c$ :

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

Thus, we can compute the angular bounds (defined by their tangent-space center and extent) from:

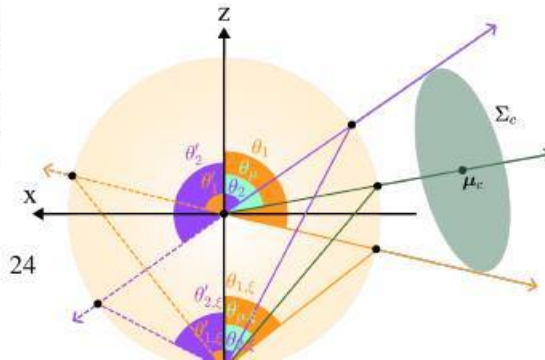
$$\begin{aligned} \left( \frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left( \frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

If the discriminant of either quadratic is negative—signaling that the camera's optical center lies inside the Gaussian ellipsoid and the angular bounds are undefined—we conservatively clamp the frustum angles to the camera's maximum field of view (FoV), a fallback that keeps the procedure well defined even for the most extreme lenses.

Note that this result is computed directly from the true 3D view-space covariance, unlike methods such as EWA and UT that depend on intermediate conic approximations in screen space. The PBF formulation thus offers exactness and efficiency together, and does so uniformly across camera models.

## D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We uniformly sample spherical angles within the maximum field of view (FoV). The resulting  $x$ ,  $y$ , and  $z$  coordinates—*i.e.*, the ray direction vectors  $\mathbf{d}$ —are projected onto a unit sphere and then mapped into pixel space for inverse color interpolation (see Sec. E). These sampled rays are subsequently grouped by their corresponding CSFs for Gaussian association.



For the many scenes captured with a pinhole camera of limited FoV, the PBF angular bounds typically lie within  $[-\frac{\pi}{2}, \frac{\pi}{2}]$ . Since  $\tan$  increases monotonically over this interval, the PBF-CSF intersections can be found by directly comparing the  $\tan$  values of their respective angular bounds. As the FoV widens, however, even when the incidence angle formed by the Gaussian centers stays within  $[-\frac{\pi}{2}, \frac{\pi}{2}]$ , the PBF angular bounds may fall outside the monotonic region of  $\tan$ . There, directly comparing  $\tan$  values can produce incorrect PBF-CSF intersections—for instance, mistakenly matching angular bounds as  $[\theta'_2, \theta_1]$  (see Fig. D.1 for an example). Handling this case is what allows the same association machinery to extend cleanly from limited-FoV to wide-FoV and omnidirectional cameras.

To recover the correct intersections in general, we apply a mirror transformation to the tangent values of both the PBF angular bounds and the CSF partitions. Drawing on the CMEI camera model (Mei & Rives 2007), we shift the optical center by a mirror parameter  $\xi$ , giving the transformation:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where  $\theta_\xi$  represents the mirror-transformed angle of  $\theta$  on the  $xz$ -plane, using a mirror parameter  $\xi$ . We set  $\xi = 1$  throughout all experiments.

Given a pair of tangent values  $[\tan \theta_1, \tan \theta_2]$  from our PBF solver (see Eq. D.6), this transformation produces four candidate mirror angles, as illustrated in Fig. D.1. The Gaussian center, by contrast, maps to a unique mirror-transformed angle  $\theta_{\mu, \xi}$ . We exploit this asymmetry by picking the two candidate angles closest to  $\theta_{\mu, \xi}$ ; these define the effective lower and upper PBF bounds  $[\theta_{1, \xi}, \theta_{2, \xi}]$ , which are then compared against CSF partitions to yield the correct intersections.

This device guarantees robust and consistent angular comparisons even beyond the monotonic domain of  $\tan$ . As reported in our timing analysis (Tab. K.5), the pre-processing pipeline—covering the mirror-angle PBF-CSF intersection computation and the canonical transformation for all Gaussians—takes under 0.13 ms in total. This confirms that the approach stays efficient even when scaled to thousands of Gaussians across wide-FoV scenarios.

### D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen space coordinates:

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation recovers exactly the expressions in 2DGS (Huang et al. 2024a) and HTGS (Hahlbohm et al. 2025). This is expected: the perspective case is simply one instance of our more general angular-domain formulation.

Concretely, 2DGS and HTGS begin from a screen-space or tangent-plane parameterization and therefore inherit assumptions that hold only for perspective (pinhole) cameras—e.g., geometric identities that arise when rays are implicitly tied to a planar,  $z$ -parallel image plane. Such an assumption does not carry over to fisheye or omnidirectional systems, where the corresponding surface in screen space is curved. Our derivation, by contrast, starts from a camera-agnostic angular parameterization in 3D and directly parameterizes Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). These weaker starting assumptions change the quantities and constraints available during derivation, and with them the conditions under which closed-form bounds and culling guarantees remain valid—conditions that, in our case, hold for generic cameras rather than pinhole ones alone.

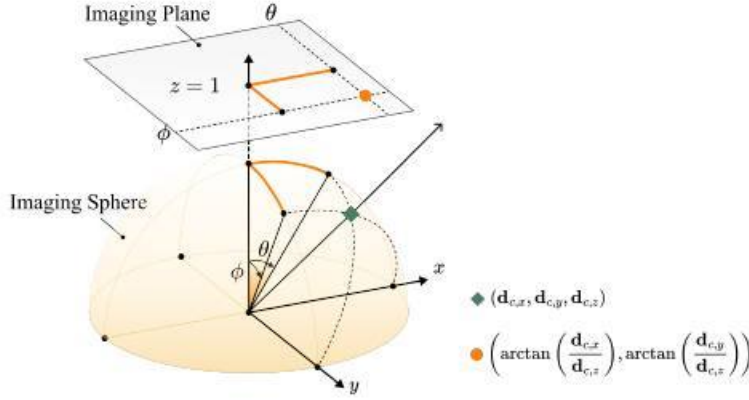


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction  $\mathbf{d}_c$  is first normalized onto the optical sphere (green spade), then angularly projected onto the  $xz$  and  $yz$  planes. The orange point marks the resulting coordinate  $(\theta, \phi)$  on the unfolded BEAP imaging plane, a construction that applies regardless of the underlying camera.

We stress that mathematics is itself a general tool; the substance of our contribution lies in deriving these bounds under weaker assumptions (without relying on screen-space structure) and validating them experimentally across both pinhole and wide-FoV camera families.

This difference in parameterization also propagates to the downstream computational pipeline. Gaussian association, tiling/duplication, and sorting all depend on how Gaussians map to tiles or bins, and an angular (CSF-based) domain yields a bucketing structure distinct from pinhole screen-space tiling. This matters in practice for large-FoV scenes: under generic cameras a Gaussian’s bound may appear partly “behind” the camera in screen coordinates (see Fig. D.1), leading screen-space methods to produce mirrored or otherwise incorrect tangent-based bounds (dashed line in Fig. D.1). To handle such cases robustly, we introduce mirror parameters in the association step (Section D.2) and fold the Gaussian center into sorting and correction—components that a perspective-only derivation never needs.

## E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To prepare training data, we sample rays that comprehensively cover the original image FoVs and feed them as input to our CUDA renderer. In parallel, we interpolate the ground-truth color for each input ray to provide accurate and uniform supervision during training.

Projecting the sampled rays onto the observed images begins by turning the incidence-angle projections  $\theta$  and  $\phi$  into normalized ray direction vectors  $\mathbf{d} = (x, y, z)^\top$ :

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

We support several camera models for the subsequent step of projecting the normalized rays  $(x, y, z)^\top$  into pixel space  $(x_p, y_p, 1)^\top$ , so that the same sampling front-end serves a range of lenses.

**Pinhole Camera Model.** For pinhole camera models, we begin from  $\frac{x}{z} = \tan \theta$  and  $\frac{y}{z} = \tan \phi$ , which gives:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

**KB Fisheye Model.** For the KB fisheye family, we derive the incidence angle  $\varphi$  from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

and the corresponding fisheye-distorted angle:

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

The rays are then projected onto pixel space, enabling inverse color interpolation from:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x\varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y\varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

## F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We give a high-level overview of our volumetric rendering pipeline (see Alg. 1 and Alg. 2). For clarity, we mirror the structure of 3DGS [Kerbl et al., 2023] to make the differences legible at a glance: newly introduced or replaced modules are highlighted in **green**, and substantially modified ones in **yellow**. The pipeline itself is generic, driven only by the sampled rays and camera metadata rather than by any hard-coded projection.

---

### ALGORITHM 1: GPU Software Asso. & Render.

---

**Function** 3DGEER ( $\mu, s, q, c, \sigma, r_c, V$ ) :

```

  CullGaussian( $\mu, V$ )                                ▷ Frustum Culling
   $F \leftarrow \text{InitCSFs}(w, h, K)$                         ▷ Partition
   $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(s, q, V)$     ▷ Transform
   $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$         ▷ Sub-frustum Asso.
   $W_{pca} \leftarrow \text{PCA\_Whitening}(s, q)$                 ▷ Transform
   $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$ 
                                                         ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
  SortByKey( $\mathcal{G}_{ID}, F_K$ )                                ▷ Globally Depth Sort
   $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
   $I \leftarrow 0$                                          ▷ Init Canvas (w/ Rays)
  for all CSFs  $f \leftarrow I$  do
    for all Rays  $i \leftarrow f$  do
       $r \leftarrow \text{GetFrustumRange}(R, f)$ 
       $m_u, d_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$       ▷ Transform
       $D_{\mu, \Sigma} \leftarrow \text{Dist}(m_u, d_u)$            ▷ Mah-distance
       $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$ 
                                                         ▷ Alpha Blending
    end
  end
  return  $I$ 

```

---

## G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

**Baselines.** We compare 3DGEER against leading volumetric particle rendering methods that target projective approximation errors in 3DGS [Kerbl et al., 2023], choosing baselines so that both ends of the camera spectrum are covered. For large-FoV fisheye inputs, we evaluate against FisheyeGS [Liao et al., 2024], EVER [Mai et al., 2025], and 3DGUT [Wu et al., 2025]; in the pinhole setting, we additionally compare with 3DGS and 3DGRT [Moënné-Loccoz et al., 2024].

---

**ALGORITHM 2: 3DGEER Initialization and Training Framework**

---

$w, h$ : width and height of the training images

$K, V$ : intrinsics and extrinsic meta info.

---

$\mu \leftarrow \text{SfM Points}$

$\mathbf{s}, \mathbf{q} \leftarrow \text{InitAttributes}()$

$c, \sigma \leftarrow \text{InitCoefficients}()$

**while** not converged **do**

$\mathbf{r}_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$

$C \leftarrow \text{3DGEER}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V)$

$\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$

$\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$

**if** IsRefinementIteration( $i$ ) **then**

$\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Optimize}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma)$

**end**

**end**

---

▷ Init Gaussian Attributes

▷ Position

▷ Scaling, Rotation

▷ Color, Opacity

▷ View Rays / Colors

▷ Asso. & Render

▷ See Alg. 1

▷ Loss

▷ Backprop & Step

▷ Kerbl et al. (2023)

**Implementation Details.** For a fair comparison, we align our implementation with the original 3DGS in initialization and hyperparameters on the ScanNet++ and MipNeRF360 datasets, and adopt the training configurations of EVER and SMERF (Duckworth et al. [2024]) for all candidate methods on ZipNeRF. As with most ray-based frameworks, our method does not directly access gradients of mean2D to control densification or pruning; we instead optimize using gradients of the view-space mean3D. Unless otherwise noted, all models share consistent supervision strategies and architectural settings across datasets. For the fisheye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al. [2025]) and reproject every rendered output to the native KB-Fisheye image space (Kannala & Brandt [2006]) before evaluation. Experiments on ScanNet++ and MipNeRF360 use full-resolution images; for ZipNeRF, given its high frame count and very large fisheye resolution, we use the maximum resolution (see Tab. 3) at which all baselines run reliably on a local desktop setup.

**Central and peripheral regions in ScanNet++ evaluation.** The rays sampled in the central region follow the range:

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}],$$

while the peripheral region collects the rays sampled over the remaining angle ranges. Here  $w, h$  denote the image width and height, and  $f_x, f_y$  are the focal lengths defined in the original KB camera model. This split lets us report performance separately across the low- and high-distortion parts of each frame.

**Cross-camera generalization evaluation on ZipNeRF.** ZipNeRF provides two data sets: raw fisheye images and their corresponding undistorted frames, the latter obtained by distortion correction and central cropping. Although the frames are aligned per frame, camera poses and point clouds were reconstructed independently with COLMAP (Schönberger & Frahm [2016]) for each set. Because COLMAP’s global optimization refines each trajectory separately, a single rigid transformation cannot fully align the two pose sets and may introduce alignment errors. To keep the cross-camera evaluation reliable and free of structure-from-motion (SfM) artifacts, we always reuse the training-time camera poses at test time. In every cross-camera experiment we vary only the intrinsic parameters while holding the extrinsic poses fixed, so that the comparison isolates transfer across camera models rather than entangling it with preprocessing-induced pose inconsistencies.

## H APPENDIX: EWA SPLATTING PRELIMINARIES

**From camera coordinates to ray coordinates.** Following the coordinate conventions of EWA Splatting (Zwicker et al. [2001]), the mapping from camera (view) space points  $\mathbf{t}$  to ray space points

$\mathbf{x}$  under the projective transformation of a pinhole camera, i.e.,  $\tau: \mathbf{t} \rightarrow \mathbf{x}$ , is:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$\text{, where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

Because these mappings are non-affine, computing the integral in ray space demands either computationally expensive treatments (Westover, 1990; Mao et al., 1995) or locally approximated ones (Zwicker et al., 2001; Kerbl et al., 2023)—a difficulty that only worsens as cameras depart further from the pinhole assumption.

**Local affine approximation.** EWA splatting by Zwicker et al. (2001) introduced the local affine approximation, denoted  $\tau_k$ , which retains the linear effects and keeps the  $k$ -indexed reconstruction kernels Gaussian in 2D screen space. Defined by the first two terms of the Taylor expansion of  $\tau$  at the camera-space point  $\mathbf{t}_k$ , it estimates a ray-space point  $\mathbf{x}$  near  $\mathbf{x}_k$  as:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where  $\mathbf{J}_\tau^{\mathbf{t}_k}$  is the Jacobian matrix of partial derivatives of  $\tau$  at  $\mathbf{t}_k$ . For the  $k$ -indexed 3D Gaussian  $\mathcal{G}_{\Sigma, \mu}$  in a given camera coordinate, the EWA local affine transformation lets us write the approximated density function as a 3D Gaussian  $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$  in the ray coordinate. For  $\forall \mathbf{x}$  around  $\mathbf{x}_k$ , we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating a 3D Gaussian  $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$  along one coordinate axis yields a 2D Gaussian  $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$ , hence:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where  $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$ , and the mean  $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$ , and the  $2 \times 2$  variance matrix  $\hat{\Sigma}_k$  is easily obtained from the  $3 \times 3$  matrix  $\Sigma_k$  by skipping the third row and column.

**Splatting approximation error.** Although the ray-space density cannot be expected to remain Gaussian once a general projective transformation is applied to the original view space, the exact density can still be obtained by substitution via Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)\right), \end{aligned} \quad (\text{H.8})$$

so that the approximation error can be estimated by Monte Carlo integration of:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Note that  $x_2$  represents the dimension along the ray direction in the ray coordinate.

## I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray-tracing (e.g., 3DGRT) can in principle be projective-exact, but in practice the BVH that 3DGRT uses for intersection tests relies on an inscribed proxy icosahedron for speed—tight at the



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** Across the compared families, 3DGEER outperforms both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods on novel view synthesis under (*Left*) MipNeRF360 (pinhole), while sustaining high runtime efficiency. Larger differences are visible in the colored boxes.



Figure J.1: **Qualitative Results on the Aria Dataset.** Across both indoor and outdoor egocentric captures, our method yields sharper floor textures indoors and clearer distant tree detail outdoors.

vertices, yet with face centers lying  $\sim 20\%$  inward by the icosahedron’s geometry. Such proxy shapes can degrade intersection accuracy, yield worse accumulated gradients during density control, and ultimately produce incorrect geometry (e.g., the missing leaves in Fig. I.1). Moreover, intersection tests for far, tiny, and cluttered Gaussians are bounded by floating-point precision, which introduces noisy rendering (for instance, in Fig. 15 of the 3DGRT paper the native rendering in the leftmost block shows visible speckles across the tree and background). Even 3DGUT with its UT approximation outperforms 3DGRT on MipNeRF360. As a controlled validation, we render from the same pretrained MipNeRF360-stump model using 3DGRT’s ray-tracing and rendering engine and, separately, our 3DGEER rendering pipeline, then compare on PSNR $\uparrow$ /SSIM $\uparrow$ /LPIPS $\downarrow$ : 3DGRT—25.79/ 0.745/ 0.264; ours—26.83/ 0.774/ 0.246 (+1.04/ +0.029/ -0.018). By building a projective-exact and efficient pipeline and further optimizing it with BEAP color supervision, 3DGEER attains both markedly faster speed and better accuracy (Tab. 4).

## J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Recent egocentric devices commonly feature wide FoVs, yet existing pipelines struggle to hold reconstruction quality at full resolution or when restricted to cropped views. Low-light imaging compounds the difficulty—limited light intake, especially with small sensors and aperture-constrained lenses, further degrades the signal. This egocentric, low-light setting is precisely the kind of real-world condition where a camera-agnostic renderer should pay off.

We evaluate on the Aria egocentric dataset ( $\sim 110^\circ$  FoV, circular shape). Our method reaches an average PSNR of 33.5dB, consistently surpassing FisheyeGS (32.8dB) across all metrics (Tab. J.1). Qualitatively, our reconstructions retain sharper peripheral detail in the indoor scene (Fig. J.1 *Left*) and recover distant structures such as trees in the outdoor scene (Fig. J.1 *Right*). Tellingly, the performance gain (+0.7 dB) falls between the improvements seen on pinhole datasets (e.g., MipNeRF360: +0.3dB; ZipNeRF-cross: +2.0dB) and on strongly distorted wide-FoV datasets (e.g., ScanNet++/ZipNeRF  $180^\circ$  diagonal FoV: +0.9–4.8dB)—a smooth trend with distortion level that is consistent with Aria’s moderate FoV and reinforces that the advantage scales gracefully across camera geometries.

Table J.1: **Quantitative comparison on Aria**, spanning indoor and outdoor egocentric scenes. Higher PSNR/SSIM and lower LPIPS are better.

| Method        | Scene      |           | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ |
|---------------|------------|-----------|-----------------|-----------------|--------------------|
| 3DGEER (Ours) | livingroom | (indoor)  | <b>36.398</b>   | <b>0.933</b>    | <b>0.226</b>       |
|               | steakhouse | (outdoor) | <b>30.662</b>   | <b>0.881</b>    | <b>0.246</b>       |
| FisheyeGS     | livingroom | (indoor)  | 35.686          | 0.927           | 0.231              |
|               | steakhouse | (outdoor) | 29.989          | 0.869           | 0.261              |

## K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

### K.1 ADDITIONAL MATERIAL.

Code, video, and an interactive HTML page are provided in the Supplementary Material.

### K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6** 3DGS collapses to black owing to the numerical singularity  $\tan 90^\circ \rightarrow \infty$ , whereas our method robustly renders views out to  $180^\circ$  and reconstructs the green chair where splat-based methods (e.g., FisheyeGS) fail. In the zoom-in of Fig. 6 our method likewise preserves structures such as the corridor and fridge, table and door, or window and cabinet—cases well outside the training FoV.



Figure K.2: **Extreme-FoV Generalization.** (Left) Our method vs. 3DGS under extreme-FoV pinhole ( $170^\circ$ ) views, trained only on undistorted central regions. (Right) Zoom-out of Fig. 6 over the complete circular  $180^\circ$  FoV, where our method generalizes robustly far beyond the training conditions.

Table K.1: **MipNeRF360 Quantitative Results by Sequence.** Per-scene breakdown across the full benchmark; overall results appear in Tab. 4

| Method | Metric             | Bicycle | Bonsai | Counter | Garden | Kitchen | Stump | Flowers | Room  | Treehill |
|--------|--------------------|---------|--------|---------|--------|---------|-------|---------|-------|----------|
| Ours   | PSNR $\uparrow$    | 25.63   | 32.23  | 29.05   | 27.60  | 31.78   | 26.83 | 21.83   | 32.09 | 22.83    |
|        | SSIM $\uparrow$    | 0.774   | 0.946  | 0.910   | 0.860  | 0.931   | 0.774 | 0.613   | 0.946 | 0.637    |
|        | LPIPS $\downarrow$ | 0.223   | 0.175  | 0.195   | 0.136  | 0.120   | 0.246 | 0.340   | 0.108 | 0.344    |

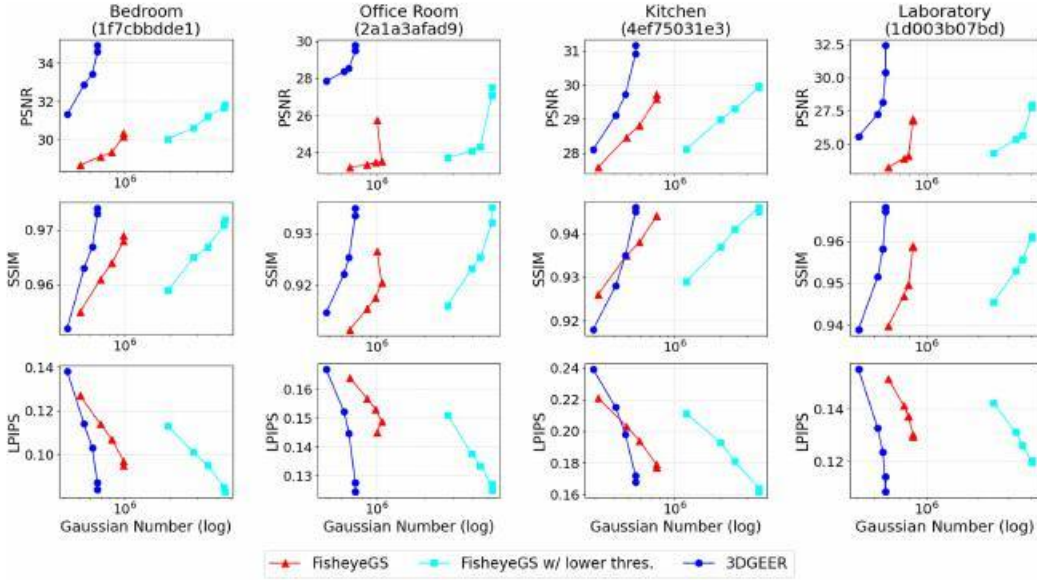


Figure K.3: **PSNR $\uparrow$  SSIM $\uparrow$  LPIPS $\downarrow$  Trends When Scaling-Up Gaussian Counts.** Across scenes and metrics, brute-force scaling fails to close the projective-exactness gap.

Table K.2: **ScanNet++ Quantitative Results by Sequence (Full FoV).** Per-scene breakdown spanning the evaluated scenes; overall results appear in Tab. 2

| Scene       | EVER  |       |       | FisheyeGS |       |       | 3DGUT        |       |       | 3DGEER (Ours) |              |              |
|-------------|-------|-------|-------|-----------|-------|-------|--------------|-------|-------|---------------|--------------|--------------|
|             | PSNR  | SSIM  | LPIPS | PSNR      | SSIM  | LPIPS | PSNR         | SSIM  | LPIPS | PSNR          | SSIM         | LPIPS        |
| Storage     | 28.24 | 0.916 | 0.164 | 24.74     | 0.932 | 0.146 | <b>29.26</b> | 0.934 | 0.147 | 29.18         | <b>0.940</b> | <b>0.141</b> |
| Office Room | 27.80 | 0.899 | 0.169 | 25.75     | 0.927 | 0.145 | 29.37        | 0.928 | 0.145 | <b>29.78</b>  | <b>0.935</b> | <b>0.125</b> |
| Bedroom     | 32.60 | 0.948 | 0.131 | 31.95     | 0.970 | 0.095 | 34.65        | 0.967 | 0.112 | <b>34.94</b>  | <b>0.974</b> | <b>0.084</b> |
| Kitchen     | 29.77 | 0.924 | 0.213 | 29.72     | 0.944 | 0.177 | 30.78        | 0.941 | 0.196 | <b>31.17</b>  | <b>0.946</b> | <b>0.168</b> |
| Laboratory  | 28.96 | 0.936 | 0.156 | 26.91     | 0.959 | 0.129 | 29.12        | 0.948 | 0.151 | <b>32.43</b>  | <b>0.968</b> | <b>0.114</b> |

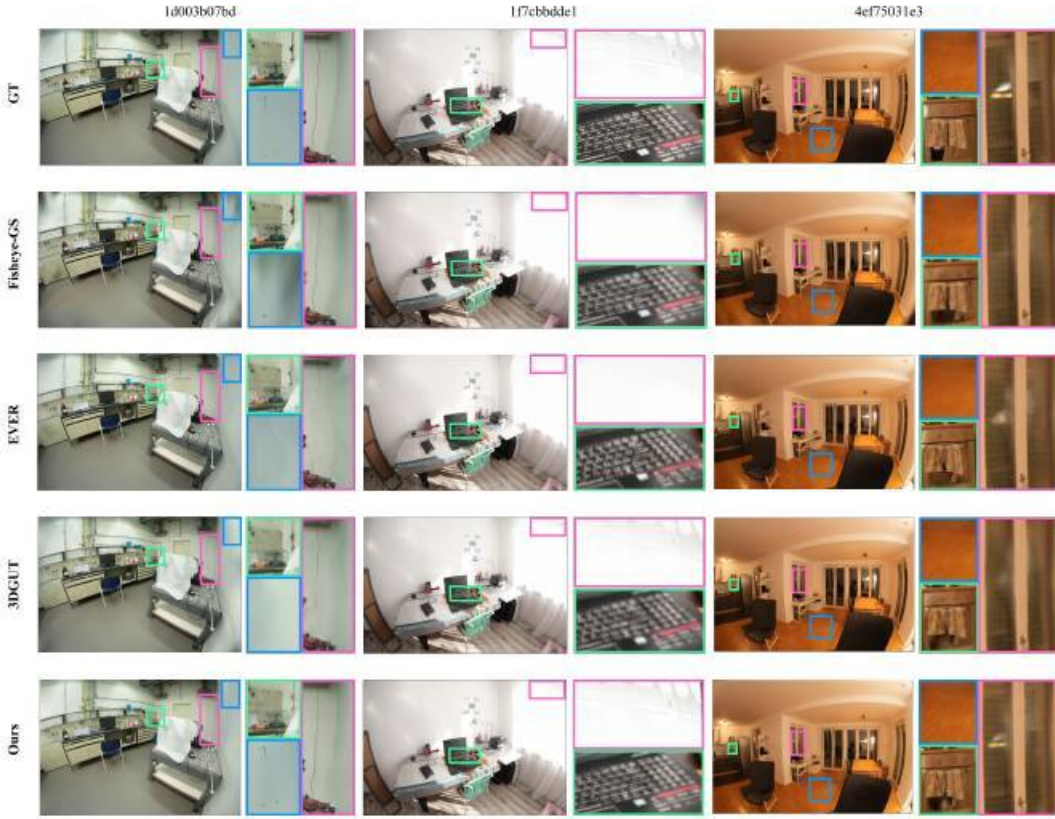


Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The largest differences appear in the zoomed-in colored boxes, especially in structural detail and artifact-prone regions. Relative to the full set of baselines, our method delivers superior peripheral reconstruction versus FisheyeGS, fewer artifacts and sharper detail than EVER, and crisper object boundaries and textures—wires, keyboards, wooden floors—than 3DGUT.

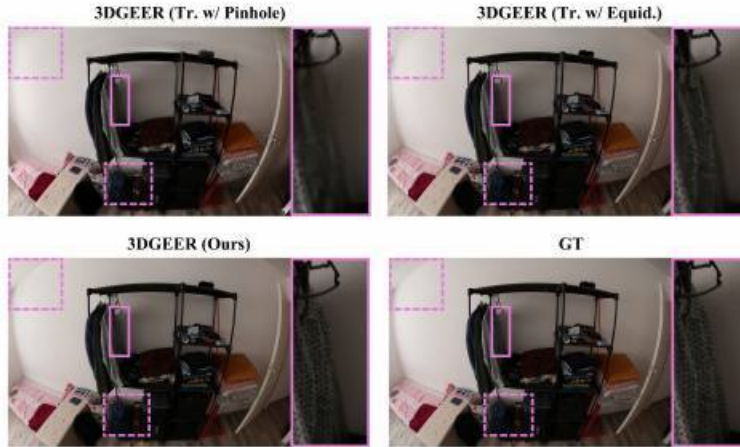


Figure K.5: **Impact of BEAP supervision.** On ScanNet++, BEAP’s balanced angular sampling yields smoother gradients and better preservation of geometric and texture detail than either pinhole or equidistant sampling. (Tab. K.8 shows the full quantitative stats).

Table K.3: **Quantitative Results on the ScanNet++**. PSNR $\uparrow$  SSIM $\uparrow$  LPIPS $\downarrow$  trends as Gaussian counts scale up, reported per scene (Bedroom, Office Room, Kitchen, and Laboratory, in order).

| Iter | FisheyeGS       |                 |                    |                  | FisheyeGS w/ Low Thres. |                 |                    |                  | 3DGEER          |                 |                    |                  |
|------|-----------------|-----------------|--------------------|------------------|-------------------------|-----------------|--------------------|------------------|-----------------|-----------------|--------------------|------------------|
|      | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$         | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ |
| 4k   | 28.67           | 0.955           | 0.127              | 512K             | 30.03                   | 0.959           | 0.113              | 1914K            | 31.33           | 0.952           | 0.138              | 422K             |
| 7k   | 29.12           | 0.961           | 0.114              | 695K             | 30.61                   | 0.965           | 0.101              | 2829K            | 32.87           | 0.963           | 0.114              | 543K             |
| 10k  | 29.36           | 0.964           | 0.107              | 827K             | 31.21                   | 0.967           | 0.095              | 3489K            | 33.42           | 0.967           | 0.103              | 616K             |
| 20k  | 30.17           | 0.968           | 0.097              | 985K             | 31.68                   | 0.971           | 0.085              | 4467K            | 34.59           | 0.973           | 0.087              | 664K             |
| 30k  | 30.36           | 0.969           | 0.095              | 985K             | 31.84                   | 0.972           | 0.083              | 4535K            | 34.94           | 0.974           | 0.084              | 664K             |

| Iter | FisheyeGS       |                 |                    |                  | FisheyeGS w/ Low Thres. |                 |                    |                  | 3DGEER          |                 |                    |                  |
|------|-----------------|-----------------|--------------------|------------------|-------------------------|-----------------|--------------------|------------------|-----------------|-----------------|--------------------|------------------|
|      | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$         | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ |
| 4k   | 23.19           | 0.911           | 0.164              | 677K             | 23.72                   | 0.916           | 0.151              | 2778K            | 27.84           | 0.915           | 0.167              | 480K             |
| 7k   | 23.33           | 0.916           | 0.157              | 867K             | 24.06                   | 0.923           | 0.138              | 3936K            | 28.37           | 0.922           | 0.152              | 621K             |
| 10k  | 23.45           | 0.918           | 0.153              | 983K             | 24.31                   | 0.925           | 0.133              | 4481K            | 28.54           | 0.925           | 0.145              | 668K             |
| 20k  | 23.51           | 0.921           | 0.149              | 1072K            | 27.07                   | 0.932           | 0.127              | 5272K            | 29.49           | 0.933           | 0.128              | 732K             |
| 30k  | 25.75           | 0.927           | 0.145              | 1001K            | 27.50                   | 0.935           | 0.125              | 5272K            | 29.78           | 0.935           | 0.125              | 732K             |

| Iter | FisheyeGS       |                 |                    |                  | FisheyeGS w/ Low Thres. |                 |                    |                  | 3DGEER          |                 |                    |                  |
|------|-----------------|-----------------|--------------------|------------------|-------------------------|-----------------|--------------------|------------------|-----------------|-----------------|--------------------|------------------|
|      | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$         | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ |
| 4k   | 27.58           | 0.926           | 0.221              | 314K             | 28.12                   | 0.929           | 0.211              | 1191K            | 28.11           | 0.918           | 0.239              | 292K             |
| 7k   | 28.47           | 0.935           | 0.203              | 485K             | 28.99                   | 0.937           | 0.193              | 1997K            | 29.11           | 0.928           | 0.215              | 410K             |
| 10k  | 28.82           | 0.938           | 0.194              | 588K             | 29.29                   | 0.941           | 0.181              | 2491K            | 29.73           | 0.935           | 0.198              | 476K             |
| 20k  | 29.60           | 0.944           | 0.179              | 760K             | 29.93                   | 0.946           | 0.164              | 3563K            | 30.91           | 0.945           | 0.172              | 552K             |
| 30k  | 29.72           | 0.944           | 0.177              | 760K             | 29.97                   | 0.945           | 0.162              | 3563K            | 31.17           | 0.946           | 0.168              | 552K             |

| Iter | FisheyeGS       |                 |                    |                  | FisheyeGS w/ Low Thres. |                 |                    |                  | 3DGEER          |                 |                    |                  |
|------|-----------------|-----------------|--------------------|------------------|-------------------------|-----------------|--------------------|------------------|-----------------|-----------------|--------------------|------------------|
|      | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$         | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | Gau $\downarrow$ |
| 4k   | 23.27           | 0.940           | 0.152              | 602K             | 24.32                   | 0.945           | 0.142              | 2429K            | 25.59           | 0.939           | 0.155              | 406K             |
| 7k   | 23.94           | 0.947           | 0.141              | 740K             | 25.36                   | 0.953           | 0.131              | 3258K            | 27.27           | 0.952           | 0.133              | 524K             |
| 10k  | 24.12           | 0.950           | 0.137              | 788K             | 25.65                   | 0.956           | 0.126              | 3569K            | 28.14           | 0.958           | 0.124              | 562K             |
| 20k  | 26.76           | 0.959           | 0.130              | 838K             | 27.77                   | 0.961           | 0.120              | 4035K            | 30.39           | 0.967           | 0.108              | 582K             |
| 30k  | 26.91           | 0.959           | 0.129              | 838K             | 27.97                   | 0.961           | 0.120              | 4035K            | 32.43           | 0.968           | 0.114              | 582K             |

Table K.4: **Runtime Analysis vs. Other Methods**. 3DGEER is compared against efficient splatting-based methods across both MipNeRF360 and ScanNet++, with a stage-wise and total runtime breakdown covering the pinhole and fisheye regimes.

| Dataset    | Method        | Avg. Timings (ms) $\downarrow$ |      |      |        |       |
|------------|---------------|--------------------------------|------|------|--------|-------|
|            |               | Prep.                          | Dup. | Sort | Render | Total |
| MipNeRF360 | 3DGS          | 0.32                           | 0.27 | 0.65 | 1.40   | 2.92  |
|            | 3DGEER (Ours) | 0.37                           | 0.17 | 0.27 | 2.10   | 3.06  |
| ScanNet++  | FisheyeGS     | 0.10                           | 0.63 | 1.45 | 2.33   | 4.70  |
|            | 3DGEER (Ours) | 0.13                           | 0.26 | 0.59 | 2.89   | 3.98  |

Table K.5: **Runtime Analysis: Asso. Method on ScanNet++**. We compare the efficiency of PBF-based association against the splatting-based EWA and UT, including the SnugBox acceleration option, all integrated into the common 3DGEER pipeline for an apples-to-apples comparison.

| 3DGEER<br>Asso. | Prep. | Avg. Timings (ms) ↓ |             |             |             | Avg.<br>FPS ↑ |
|-----------------|-------|---------------------|-------------|-------------|-------------|---------------|
|                 |       | Dup.                | Sort        | Render      | Total       |               |
| EWA             | 0.13  | 1.17                | 3.27        | 12.69       | 17.84       | 56            |
| + SnugBox       | 0.13  | 0.60                | 1.42        | 6.22        | 8.68        | 115           |
| UT              | 0.13  | 0.71                | 1.94        | 7.97        | 11.16       | 90            |
| + SnugBox       | 0.13  | 0.28                | 0.64        | 3.16        | 4.38        | 228           |
| PBF (Ours)      | 0.13  | <b>0.26</b>         | <b>0.59</b> | <b>2.89</b> | <b>3.98</b> | <b>251</b>    |

Table K.6: **Impact of PBF-CSF Association**. The evaluation uses the artifact-sensitive LPIPS metric. To probe cross-model robustness, each variant is additionally tested on models trained under the other association strategies. We further report the rendering-time memory cost, the average number of Gaussians associated per tile, and the performance gap  $\Delta$ LPIPS between in-method and cross-method evaluation, quantifying both consistency and how well each scheme generalizes across training pipelines.

| Asso. Method | Gaus. ↓ per Tile |            | Mem. ↓<br>(GB) | LPIPS ↓       | Cross-LPIPS ↓ | $\Delta$ LPIPS ↓<br>(1e-2) |
|--------------|------------------|------------|----------------|---------------|---------------|----------------------------|
|              | mean             | std        |                |               |               |                            |
| EWA          | 2203             | 1447       | 2.2            | 0.1250        | 0.1321        | 0.71                       |
| UT           | 1377             | 1058       | 1.4            | 0.1249        | 0.1302        | 0.53                       |
| PBF (Ours)   | <b>475</b>       | <b>427</b> | <b>0.63</b>    | <b>0.1245</b> | <b>0.1251</b> | <b>0.06</b>                |

Table K.7: **Training-Time Transmittance Func. Replacement**. Per-scene breakdown; the overall results appear in Tab. 5.

| Scene       | 3DGS Splats + PBF |       |       |               | FisheyeGS Splats + PBF |       |       |               | 3DGEER (Ours) |       |       |               |
|-------------|-------------------|-------|-------|---------------|------------------------|-------|-------|---------------|---------------|-------|-------|---------------|
|             | PSNR              | SSIM  | LPIPS | Gaus. Num (k) | PSNR                   | SSIM  | LPIPS | Gaus. Num (k) | PSNR          | SSIM  | LPIPS | Gaus. Num (k) |
| Storage     | 19.27             | 0.859 | 0.173 | 1293          | 24.75                  | 0.933 | 0.150 | 781           | 29.18         | 0.940 | 0.141 | 429           |
| Office Room | 20.89             | 0.848 | 0.182 | 1527          | 25.89                  | 0.928 | 0.150 | 1119          | 29.78         | 0.935 | 0.125 | 732           |
| Bedroom     | 27.39             | 0.923 | 0.119 | 1542          | 31.96                  | 0.970 | 0.096 | 1079          | 34.94         | 0.974 | 0.084 | 664           |
| Kitchen     | 25.15             | 0.890 | 0.239 | 1241          | 29.87                  | 0.949 | 0.182 | 801           | 31.17         | 0.946 | 0.168 | 552           |
| Laboratory  | 21.60             | 0.869 | 0.173 | 1379          | 27.03                  | 0.962 | 0.131 | 823           | 32.43         | 0.968 | 0.114 | 582           |

Table K.8: **Impact of Different Supervision Space**. Per-scene breakdown across supervision spaces; the overall results appear in Tab. 6.

| Supervision Space<br>Scene | 3DGEER w/o BEAP<br>Central Perspective |       |       | 3DGEER w/o BEAP<br>Perspective |       |       | 3DGEER w/o BEAP<br>Equidistant |       |       | 3DGEER (Ours)<br>BEAP |       |       |
|----------------------------|----------------------------------------|-------|-------|--------------------------------|-------|-------|--------------------------------|-------|-------|-----------------------|-------|-------|
|                            | PSNR                                   | SSIM  | LPIPS | PSNR                           | SSIM  | LPIPS | PSNR                           | SSIM  | LPIPS | PSNR                  | SSIM  | LPIPS |
| Storage                    | 28.42                                  | 0.931 | 0.143 | 19.74                          | 0.834 | 0.302 | 29.03                          | 0.938 | 0.142 | 29.18                 | 0.940 | 0.141 |
| Office Room                | 28.90                                  | 0.927 | 0.133 | 21.12                          | 0.825 | 0.291 | 29.71                          | 0.932 | 0.126 | 29.78                 | 0.935 | 0.125 |
| Bedroom                    | 33.86                                  | 0.965 | 0.090 | 24.38                          | 0.882 | 0.240 | 34.60                          | 0.973 | 0.096 | 34.94                 | 0.974 | 0.084 |
| Kitchen                    | 30.65                                  | 0.941 | 0.172 | 23.23                          | 0.889 | 0.338 | 31.23                          | 0.941 | 0.177 | 31.17                 | 0.946 | 0.168 |
| Laboratory                 | 27.35                                  | 0.951 | 0.118 | 17.08                          | 0.834 | 0.329 | 30.65                          | 0.956 | 0.135 | 32.43                 | 0.968 | 0.114 |

**Table K.9: ZipNeRF Quantitative Results by Sequence.** Per-scene breakdown (Alameda, Berlin, London, and NYC, in order) across the cross-camera settings; the overall results appear in Tab. 3

| Train Data | 1/8 FE          |        | 1/8 FE          |        | 1/8 FE             |        | 1/4 PH          |        | 1/4 PH          |        | 1/4 PH             |        |
|------------|-----------------|--------|-----------------|--------|--------------------|--------|-----------------|--------|-----------------|--------|--------------------|--------|
| Test Data  | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH |
| Method     | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        |
| FisheyeGS  | 20.74           | 23.64  | 0.820           | 0.802  | 0.223              | 0.277  | 17.39           | 23.74  | 0.736           | 0.839  | 0.265              | 0.240  |
| EVER       | 22.46           | 23.06  | 0.852           | 0.777  | 0.144              | 0.274  | 20.06           | 22.83  | 0.789           | 0.794  | 0.213              | 0.250  |
| 3DGUT      | 21.78           | 22.39  | 0.841           | 0.718  | 0.193              | 0.368  | 17.03           | 22.58  | 0.588           | 0.765  | 0.329              | 0.311  |
| Ours       | 24.60           | 25.00  | 0.857           | 0.839  | 0.154              | 0.241  | 21.49           | 25.21  | 0.792           | 0.807  | 0.232              | 0.277  |

| Train Data | 1/8 FE          |        | 1/8 FE          |        | 1/8 FE             |        | 1/4 PH          |        | 1/4 PH          |        | 1/4 PH             |        |
|------------|-----------------|--------|-----------------|--------|--------------------|--------|-----------------|--------|-----------------|--------|--------------------|--------|
| Test Data  | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH |
| Method     | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        |
| FisheyeGS  | 24.25           | 28.25  | 0.895           | 0.924  | 0.206              | 0.209  | 20.08           | 27.86  | 0.848           | 0.929  | 0.233              | 0.194  |
| EVER       | 25.26           | 26.97  | 0.912           | 0.907  | 0.155              | 0.221  | 23.18           | 27.72  | 0.877           | 0.913  | 0.196              | 0.212  |
| 3DGUT      | 25.37           | 27.02  | 0.904           | 0.876  | 0.197              | 0.288  | 18.57           | 27.43  | 0.730           | 0.898  | 0.297              | 0.249  |
| Ours       | 25.51           | 29.15  | 0.916           | 0.925  | 0.166              | 0.194  | 23.58           | 29.51  | 0.878           | 0.915  | 0.219              | 0.228  |

| Train Data | 1/8 FE          |        | 1/8 FE          |        | 1/8 FE             |        | 1/4 PH          |        | 1/4 PH          |        | 1/4 PH             |        |
|------------|-----------------|--------|-----------------|--------|--------------------|--------|-----------------|--------|-----------------|--------|--------------------|--------|
| Test Data  | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH |
| Method     | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        |
| FisheyeGS  | 23.57           | 26.65  | 0.829           | 0.861  | 0.253              | 0.254  | 20.83           | 27.33  | 0.768           | 0.883  | 0.288              | 0.229  |
| EVER       | 24.59           | 26.83  | 0.823           | 0.859  | 0.218              | 0.223  | 23.14           | 26.98  | 0.793           | 0.860  | 0.268              | 0.235  |
| 3DGUT      | 25.61           | 26.25  | 0.854           | 0.803  | 0.221              | 0.336  | 19.82           | 26.41  | 0.669           | 0.833  | 0.313              | 0.291  |
| Ours       | 26.60           | 27.84  | 0.895           | 0.882  | 0.142              | 0.230  | 23.95           | 27.15  | 0.840           | 0.846  | 0.217              | 0.289  |

| Train Data | 1/8 FE          |        | 1/8 FE          |        | 1/8 FE             |        | 1/4 PH          |        | 1/4 PH          |        | 1/4 PH             |        |
|------------|-----------------|--------|-----------------|--------|--------------------|--------|-----------------|--------|-----------------|--------|--------------------|--------|
| Test Data  | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE          | 1/4 PH | 1/8 FE             | 1/4 PH |
| Method     | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        | PSNR $\uparrow$ |        | SSIM $\uparrow$ |        | LPIPS $\downarrow$ |        |
| FisheyeGS  | 24.15           | 27.21  | 0.890           | 0.885  | 0.162              | 0.215  | 19.41           | 27.50  | 0.811           | 0.907  | 0.203              | 0.183  |
| EVER       | 28.39           | 27.71  | 0.931           | 0.860  | 0.095              | 0.228  | 25.22           | 28.27  | 0.879           | 0.883  | 0.154              | 0.192  |
| 3DGUT      | 26.31           | 26.69  | 0.917           | 0.817  | 0.121              | 0.302  | 19.00           | 27.42  | 0.659           | 0.866  | 0.263              | 0.228  |
| Ours       | 28.26           | 28.49  | 0.922           | 0.906  | 0.097              | 0.190  | 24.52           | 28.55  | 0.874           | 0.885  | 0.168              | 0.220  |